

PostgreSQL и SQLAlchemy 2.0

Полное руководство: как устроена база, как её описать в Python, как хранить, связывать, запрашивать и мигрировать данные.

Все примеры — рабочий код проекта `nedra`.

Как читать. Часть I — сама база: типы, ограничения, индексы, SQL. Части II–IV — SQLAlchemy: модели, связи, запросы. Часть V — движок, сессии и стык с FastAPI. Часть VI — миграции Alembic и тесты. Часть VII — практика и шпаргалка. Часть VIII — пятьдесят вопросов с собеседований с короткими ответами. Каждый импорт объясняется в месте первого использования, а глава 39 собирает их в одну карту.

Содержание

Часть I. PostgreSQL

1. Что такое СУБД и зачем нам PostgreSQL
2. Как поднять PostgreSQL и подключиться
3. Таблицы и типы данных
4. Ограничения: как база защищает данные
5. Индексы: как база ищет быстро
6. SELECT: читаем данные
7. JOIN: соединяем таблицы
8. Агрегаты и группировка
9. Подзапросы, CTE и оконные функции
10. INSERT, UPDATE, DELETE
11. Транзакции
12. Эксплуатация: бэкапы, роли, часовые пояса

Часть II. Основы SQLAlchemy

13. Что такое ORM и зачем он нужен
14. Три кирпича: Base, Mapped, mapped_column
15. metadata — реестр схемы
16. Типы колонок
17. Параметры колонок: default, server_default, onupdate
18. Ограничения и индексы: __table_args__
19. Enum в моделях

20. Миксины: переиспользование колонок

Часть III. Связи

21. Как понять, какая связь нужна

22. Один ко многим (1:N)

23. Один к одному (1:1)

24. Многие ко многим (M:N)

25. Промежуточная таблица: Table или модель

26. Параметры relationship

27. Загрузка данных: lazy, N+1 и async

28. Удаление: cascade и ondelete

Часть IV. Запросы

29. select: основы

30. Операторы и условия

31. Функции SQL: func

32. Агрегация и группировка

33. JOIN: соединение таблиц

34. Подзапросы, CTE и exists

35. Изменение данных: insert, update, delete, UPSERT

36. Сессия: flush, commit, refresh

Часть V. Движок, сессии и FastAPI

37. Engine: соединения, пул и async

38. Сессия: жизненный цикл, identity map, expire_on_commit

39. Карта импортов: что откуда и почему

Часть VI. Миграции Alembic

40. Зачем миграции и как Alembic устроен

41. Настройка: alembic.ini и env.py

42. Пишем миграцию руками

43. Autogenerate и alembic check

44. Эксплуатация миграций

45. Тестирование: SQLite, TestClient и подмена сессии

Часть VII. Практика

46. Как проектировать схему

47. Частые ошибки

48. Шпаргалка

Часть VIII. Вопросы с собеседований

49. Топ-50 вопросов и коротких ответов

Часть I. PostgreSQL

SQLAlchemy — это прослойка между Python и базой. Чтобы понимать, что она делает и почему иногда ругается, нужно понимать саму базу. Поэтому начинаем с PostgreSQL: как он устроен, какие у него типы, ограничения, индексы и как выглядит SQL, который SQLAlchemy генерирует за вас.

1. Что такое СУБД и зачем нам PostgreSQL

База данных — это файлы на диске, в которых лежат таблицы. Система управления базами данных (СУБД) — это программа, которая эти файлы читает и пишет, следит за целостностью, разруливает одновременные запросы и отвечает на SQL. PostgreSQL — одна из таких программ. Она работает как отдельный сервер: постоянно запущена, слушает порт (по умолчанию 5432) и ждёт подключений.

Клиент — это любая программа, которая подключается к серверу и отправляет SQL: консольная утилита `psql`, `DBeaver`, ваш `FastAPI`-бэкенд через драйвер `asyncpg`. Клиент открывает соединение, отправляет текст запроса, получает строки в ответ. SQLAlchemy — клиентская библиотека: она собирает SQL, отправляет его через драйвер и превращает ответ в объекты Python.

Как устроено внутри

- **Кластер** — один запущенный сервер PostgreSQL. В Docker это один контейнер.
- **База данных** — изолированное пространство внутри кластера. У нас одна база, имя берётся из переменной `POSTGRES_DB`. Подключение всегда идёт к конкретной базе.
- **Схема** — папка внутри базы для группировки таблиц. По умолчанию всё лежит в схеме `public`, и её имя можно не писать: `articles` значит `public.articles`.
- **Таблица** — набор строк с фиксированными колонками. `articles`, `tags`, `article_tags`.
- **Роль** — пользователь или группа. У нас одна роль из `POSTGRES_USER`, она владелец базы и может всё.

Почему не SQLite

SQLite — это библиотека, а не сервер: база лежит в одном файле, никакого отдельного процесса нет. Для тестов это удобно, и наш `e2e`-скрипт гоняет бэкенд именно на SQLite. Но на проде у нас параллельные запросы от сайта, админки и скрипта загрузки статей. SQLite пишет по одному писателю за раз и держит блокировку на весь файл, PostgreSQL умеет писать в разные строки одновременно. Плюс типы: в SQLite колонка объявлена как `TIMESTAMP`, а хранит строку, отсюда наши приключения со сравнением дат в тестах. PostgreSQL проверяет типы строго.

	SQLite	PostgreSQL
Где живёт	файл рядом с приложением	отдельный сервер, свой контейнер
Параллельная запись	один писатель	много писателей, блокировки по строкам

Диалект и драйвер — разные вещи. Диалект `postgresql` говорит SQLAlchemy, какие особенности SQL использовать: `RETURNING`, `JSONB`, `ON CONFLICT`. Драйвер — это Python-библиотека, которая физически открывает сокет и передаёт байты. Для синхронного кода это `psycopg2` или `psycopg`, для асинхронного — `asyncpg`. Драйвер ставится отдельно, поэтому в `requirements.txt` лежат и `sqlalchemy[asyncio]`, и `asyncpg`. Alembic использует тот же URL, поэтому у него в `env.py` тоже асинхронный движок.

Консоль psql

Когда нужно посмотреть данные глазами или выполнить ручной запрос, заходим в `psql` внутри контейнера базы. Переменные окружения там уже есть, поэтому пароль вводить не нужно.

Полезные команды psql

```
cd /opt/nedra
docker compose exec -it db sh -c 'psql -U "$POSTGRES_USER" -d "$POSTGRES_DB"'

nedra=# \dt                -- список таблиц
nedra=# \d articles        -- структура таблицы: колонки, индексы, внешние ключи
nedra=# \di                -- список индексов
nedra=# SELECT count(*) FROM articles WHERE section = 'news';
nedra=# \x                 -- широкие строки показывать вертикально
nedra=# \q                 -- выйти
```

Команды с обратным слэшем — это команды самой утилиты `psql`, не SQL. Сервер их не видит. Всё остальное — обычный SQL, и он обязан заканчиваться точкой с запятой, иначе `psql` ждёт продолжения строки.

3. Таблицы и типы данных

Таблица создаётся командой `CREATE TABLE`. Вот как выглядит наша таблица статей, если попросить PostgreSQL показать её (`\d articles`) или сгенерировать миграцией. Это тот SQL, который SQLAlchemy построил по нашей модели.

Таблица articles в чистом SQL

```
CREATE TABLE articles (
  id          SERIAL PRIMARY KEY,
  slug        VARCHAR(255) NOT NULL UNIQUE,
  section     VARCHAR(16)  NOT NULL DEFAULT 'blog',
  title       VARCHAR(255) NOT NULL,
  description  VARCHAR(400),
  cover_image VARCHAR(500),
  content     TEXT         NOT NULL,
  toc         JSONB,
  seo_title   VARCHAR(255),
  seo_description VARCHAR(300),
  seo_keywords VARCHAR(500),
  views_count INTEGER      NOT NULL,
  likes_count  INTEGER      NOT NULL,
  dislikes_count INTEGER    NOT NULL,
  published_at TIMESTAMP WITH TIME ZONE,
  created_at   TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT now(),
  updated_at   TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT now()
);
CREATE INDEX ix_articles_section ON articles (section);
CREATE INDEX ix_articles_published_at ON articles (published_at);
```

Каждая колонка — это имя, тип и набор ограничений. Тип говорит, что можно хранить и сколько это занимает; ограничения — что база отклонит.

Числа

Тип	Диапазон	Когда брать
smallint	±32 тысячи, 2 байта	маленькие коды: значение реакции –1/1
integer	±2,1 миллиарда, 4 байта	идентификаторы, счётчики — наш выбор по умолчанию
bigint	±9·10 ¹⁸ , 8 байт	id в таблицах с миллиардами строк, суммы в копейках
numeric(p, s)	точная десятичная дробь	деньги, проценты — там, где 0,1 + 0,2 должно быть ровно 0,3
real / double	плавающая точка	физика, координаты; для денег нельзя
serial / identity	integer с автонумерацией	первичные ключи

SERIAL — это сокращение: PostgreSQL создаёт отдельный объект-последовательность `articles_id_seq` и ставит колонке `DEFAULT nextval('articles_id_seq')`. Каждая вставка берёт следующий номер. Номера не переиспользуются: удалили статью с id 7 — следующая получит 8, а не 7. Откатившаяся транзакция тоже «сжигает» номер, поэтому дырки в id нормальны. SQLAlchemy делает SERIAL сам для Integer, `primary_key=True`.

Строки

VARCHAR(n) — строка не длиннее n символов, база откажет при превышении. TEXT — строка любой длины. Внутри PostgreSQL они хранятся одинаково и работают одинаково быстро, разница только в проверке длины. Правило: ограничение длины ставим там, где оно осмысленно (slug, заголовок, meta description для поисковика), для тела статьи берём TEXT. Кодировка базы UTF-8, кириллица, эмодзи и китайские иероглифы хранятся без танцев.

Дата и время

Здесь чаще всего ошибаются, поэтому подробно. В PostgreSQL два типа временных меток:

- `timestamp` (без часового пояса) хранит «настенные часы»: 9 сентября 2026, 09:00. Чьи это 09:00 — новосибирские, московские — база не знает.
- `timestampz` (`TIMESTAMP WITH TIME ZONE`) хранит момент времени: внутри всегда UTC, а при показе переводит в часовой пояс сессии. 09:00 Новосибирска и 05:00 Москвы — это один и тот же момент, и он сохранится одинаково.

Мы везде используем `timestampz`, в SQLAlchemy это `DateTime(timezone=True)`. Поэтому скрипт загрузки статей отправляет дату с явным смещением `2026-09-07T09:00:00+07:00`, а сравнение `published_at <= now()` работает корректно независимо от того, в какой зоне живёт сервер. `now()` — функция базы, возвращает момент начала текущей транзакции. Отдельные типы `date` (только день) и `time` (только время) используем реже.

Логический тип и NULL

`boolean` хранит `true/false`. И у любого типа есть третье состояние — `NULL`, «значения нет». `NULL` — не ноль и не пустая строка. Любое сравнение с `NULL` даёт «неизвестно», поэтому `WHERE published_at = NULL` никогда ничего не вернёт; нужно писать `IS NULL` или `IS NOT NULL`. Именно так устроен наш черновик: у неопубликованной статьи `published_at IS NULL`.

JSON и JSONB

`json` хранит текст как есть, `jsonb` разбирает его в бинарное дерево: занимает чуть больше при записи, но по нему можно искать и строить индексы. Мы храним оглавление статьи в `JSONB`: список объектов `{"id": ..., "title": ...}`. Достать из него что-то прямо в SQL можно операторами `->` (вернуть JSON) и `->>` (вернуть текст).

Первый заголовок оглавления у статей с большим оглавлением

```
SELECT title, toc->0->>'title' AS first_section
FROM articles
WHERE jsonb_array_length(toc) > 3;
```

UUID и массивы

`uuid` — 128-битный случайный идентификатор вида `651458c3-d614-45df-ad84-e3901af1ac64`. У нас так устроен `visitor_id` в cookie; мы храним его как `VARCHAR(36)`, потому что SQLite такого типа не знает, а тесты гоняем на SQLite. В чистом PostgreSQL-проекте лучше брать настоящий `uuid`: он занимает 16 байт вместо 36 и валидируется базой. Массивы (`text[]`, `integer[]`) PostgreSQL тоже умеет, но для связей «многие ко многим» они плохая замена отдельной таблице: по ним нет внешних ключей.

4. Ограничения: как база защищает данные

Ограничение (`constraint`) — правило, которое база проверяет при каждой вставке и обновлении. Нарушили — получите ошибку, транзакция откатится. Это последняя линия обороны: валидация в Pydantic защищает API, а ограничение в базе защищает данные от всех сразу — от миграций, скриптов, ручных правок в `rsql` и от кода, который напишут после вас.

PRIMARY KEY — первичный ключ

Уникальный и не `NULL` идентификатор строки. Обычно это `id SERIAL`. Но ключ может быть составным: у `article_reactions` первичный ключ — пара (`article_id`, `visitor_id`). Так база гарантирует «один посетитель — одна реакция на статью» без отдельного уникального индекса, а поиск по паре идёт по индексу первичного ключа. В SQLAlchemy это два поля с `primary_key=True`.

UNIQUE — уникальность

Две строки не могут иметь одно значение. `slug` у статьи уникален, потому что это адрес страницы. Сервис перед вставкой подбирает свободный `slug`, но проверка в коде не спасает от гонки: два запроса одновременно проверили «свободно» и оба вставили. `UNIQUE` в базе отклонит второго, и мы поймем `IntegrityError`. `NULL` в уникальной колонке допускается сколько угодно раз: `NULL` не равен `NULL`.

NOT NULL — обязательность

Колонка обязана иметь значение. SQLAlchemy выводит это из аннотации: `Mapped[str]` даёт `NOT NULL`, `Mapped[str | None]` разрешает `NULL`. Поэтому у нас `description` и `cover_image` необязательны, а `title` и `content` обязательны.

CHECK — произвольная проверка

Ограничение из нашей модели ArticleReaction

```
ALTER TABLE article_reactions
    ADD CONSTRAINT ck_article_reactions_value CHECK (value IN (-1, 1));
```

Любое логическое выражение над колонками строки. У нас `value IN (-1, 1)`: лайк или дизлайк, ничего третьего. Именно поэтому в модели появился `__table_args__`: параметров колонки для такого правила нет, оно относится к таблице. Другие типичные проверки: `price >= 0`, `char_length(slug) > 0`, `ends_at > starts_at`.

FOREIGN KEY — внешний ключ

Значение колонки должно существовать в другой таблице. `article_tags.article_id` обязан указывать на реальную статью. Без внешнего ключа в базе можно оставить «висячие» связи на удалённые статьи. Внешний ключ несёт и правило поведения при удалении родителя:

ON DELETE	Что произойдёт при удалении статьи	Где у нас
RESTRICT / NO ACTION	удаление запрещено, пока есть дочерние строки	по умолчанию, если ничего не указать
CASCADE	дочерние строки удаляются вместе с родителем	<code>article_tags</code> , <code>article_views</code> , <code>article_reactions</code>
SET NULL	внешний ключ обнуляется, строка остаётся	заказ без исполнителя после удаления пользователя
SET DEFAULT	подставляется значение по умолчанию	редко

У нас всё каскадом: удалили статью — исчезли её связи с тегами, просмотры и реакции. Это правило базы, оно сработает даже при удалении через `psql`. У SQLAlchemy есть свой `cascade` на уровне `relationship`, он работает только когда вы удаляете через сессию; про разницу — в главе про связи.

DEFAULT — значение по умолчанию

Если при вставке колонку не указали, база подставит значение сама. `section DEFAULT 'blog'` позволило добавить колонку в уже заполненную таблицу: все старые статьи получили «blog» без единого `UPDATE`. `created_at DEFAULT now()` ставит время базы, а не Python. В SQLAlchemy это `server_default`; про отличие от `default` — в главе о параметрах колонок.

Имена ограничений

У каждого ограничения есть имя. Если его не задать, PostgreSQL придумает своё (`article_reactions_value_check`), и в миграции для удаления его придётся угадывать. Поэтому мы именуем явно: `ck_` для `CHECK`, `uq_` для `UNIQUE`, `fk_` для внешних ключей, `ix_` для индексов.

5. Индексы: как база ищет быстро

Без индекса поиск `WHERE slug = 'ovos-kogda-trebuetsya'` — это полный перебор таблицы: база читает все строки и сравнивает каждую. На ста статьях незаметно, на миллионе строк — секунды. Индекс — отдельная структура (по умолчанию B-дерево), отсортированная по значению колонки, с указателями на строки. Поиск по нему — несколько шагов по дереву вместо перебора.

Что индексируется автоматически

- PRIMARY KEY — всегда есть уникальный индекс.
- UNIQUE — тоже создаёт уникальный индекс; отдельный `index=True` на `slug` не нужен.
- FOREIGN KEY — индекс **не** создаётся. Колонки внешних ключей стоит индексировать руками: по ним постоянно идут JOIN и каскадные удаления.

Какие индексы у нас и зачем

Индексы проекта

```
CREATE INDEX ix_articles_published_at ON articles (published_at);
-- каждый публичный запрос сортирует по этой колонке и фильтрует published_at <= now()

CREATE INDEX ix_articles_section ON articles (section);
-- фильтр раздела blog / news в каждом списке

-- первичные ключи article_views и article_reactions – составные,
-- и поиск «голосовал ли посетитель X за статью Y» идёт по ним
```

Составные индексы

Индекс по нескольким колонкам полезен, когда запрос фильтрует по ним вместе. Порядок колонок важен: индекс `(section, published_at)` ускорит запрос `WHERE section = 'news' ORDER BY published_at DESC`, потому что база найдёт блок «news» и внутри него строки уже отсортированы. Индекс `(published_at, section)` для этого запроса почти бесполезен. Правило: сначала колонки с равенством, потом с диапазоном или сортировкой. Для нашего списка новостей это был бы лучший индекс, но при сотне статей разницы не заметить, поэтому оставлены два простых.

Частичные индексы и индексы по JSONB

Индексы для особых случаев

```
-- индекс только по опубликованным: черновики в списках нет, зачем их индексировать
CREATE INDEX ix_articles_published_only
  ON articles (published_at DESC)
  WHERE published_at IS NOT NULL;

-- индекс по содержимому JSONB для поиска вида toc @> '[{"id": "riski"}]'
CREATE INDEX ix_articles_toc ON articles USING GIN (toc);
```

Цена индекса

Каждый индекс ускоряет чтение и замедляет запись: при INSERT и UPDATE база обновляет все индексы таблицы. Плюс место на диске. Поэтому индексы ставят под реальные запросы, а не «на всякий случай». Проверить, используется ли индекс, можно командой EXPLAIN ANALYZE: она показывает план выполнения запроса.

Как проверить план запроса

```
EXPLAIN ANALYZE
SELECT * FROM articles
WHERE section = 'news' AND published_at <= now()
ORDER BY published_at DESC LIMIT 12;

-- Index Scan using ix_articles_published_at ...    ← индекс сработал
-- Seq Scan on articles ...                        ← полный перебор
```

На маленьких таблицах планировщик часто выбирает Seq Scan даже при наличии индекса: прочитать десять строк подряд дешевле, чем ходить по дереву. Это не ошибка. Индекс начнёт использоваться, когда данных станет много.

6. SELECT: читаем данные

Весь SQL для чтения — это одна команда с необязательными частями. Порядок частей фиксирован, и его полезно помнить, потому что SQLAlchemy повторяет его один в один.

Скелет *SELECT*

SELECT	колонки	-- что вернуть
FROM	таблица	-- откуда
JOIN	другая ON условие	-- с чем соединить
WHERE	условие	-- какие строки взять
GROUP BY	колонки	-- как сгруппировать
HAVING	условие по группам	-- какие группы оставить
ORDER BY	колонки	-- как отсортировать
LIMIT	n OFFSET m	-- сколько и с какого места

База выполняет их не в том порядке, в каком написано: сначала FROM и JOIN, потом WHERE, потом GROUP BY и HAVING, потом SELECT, потом ORDER BY и LIMIT. Отсюда правило: в WHERE нельзя использовать псевдоним из SELECT, он ещё не вычислен, а в ORDER BY — можно.

Условия WHERE

Операторы условий

SELECT slug, title FROM articles WHERE section = 'news';	-- равенство
SELECT slug FROM articles WHERE views_count >= 100 AND likes_count > 0;	-- и
SELECT slug FROM articles WHERE section = 'blog' OR likes_count > 10;	-- или
SELECT slug FROM articles WHERE published_at IS NULL;	-- черновики
SELECT slug FROM articles WHERE title ILIKE '%ростехнадзор%';	-- поиск без учёта регистра
SELECT slug FROM articles WHERE section IN ('blog', 'news');	-- один из
SELECT slug FROM articles WHERE views_count BETWEEN 10 AND 100;	-- диапазон включительно
SELECT slug FROM articles WHERE NOT (section = 'news');	-- отрицание

LIKE учитывает регистр, ILIKE — нет (это расширение PostgreSQL). Процент % — любое количество символов, подчёркивание _ — ровно один. Поиск с процентом в начале не использует обычный индекс, потому что неизвестно, с чего начинается строка.

Сортировка и порции

Третья страница новостей по 12 штук

```
SELECT slug, published_at
FROM articles
WHERE section = 'news' AND published_at <= now()
ORDER BY published_at DESC, id DESC
LIMIT 12 OFFSET 24;
```

Вторая колонка сортировки `id DESC` нужна для стабильности: если у двух статей одинаковая дата, без неё порядок между запросами может меняться, и статья «перепрыгнет» со второй страницы на третью. `OFFSET` пропускает строки, но база их всё равно читает, поэтому на глубоких страницах он медленный. Для ленты с кнопкой «Ещё» на сотнях записей это нормально; на миллионах используют курсор: `WHERE published_at < :last_seen`.

Уникальные значения и вычисления

DISTINCT, выражения, COALESCE, CASE

```
SELECT DISTINCT section FROM articles;           -- какие разделы есть
SELECT slug, likes_count - dislikes_count AS rating -- вычисляемая колонка
FROM articles ORDER BY rating DESC;
SELECT slug, COALESCE(seo_title, title) AS page_title -- первое не-NULL значение
FROM articles;
SELECT slug, CASE WHEN published_at IS NULL THEN 'черновик'
                  WHEN published_at > now() THEN 'запланирована'
                  ELSE 'опубликована' END AS status
FROM articles;
```

`COALESCE` — это ровно то, что фронтенд делает через `seo_title ?? title`: берём первое непустое. `CASE` — условное выражение, аналог `if/elif/else`. Статус «запланирована» в админке вычисляется на фронте так же.

7. JOIN: соединяем таблицы

Данные разложены по таблицам, чтобы не дублировать: название тега хранится один раз в `tags`, а связь статьи с тегом — строка в `article_tags`. Когда нужны статья и её теги вместе, таблицы соединяют. `JOIN` берёт строки одной таблицы и приставляет к ним подходящие строки другой по условию `ON`.

Статьи с тегами через промежуточную таблицу

```
SELECT a.title, t.title AS tag
FROM articles a
JOIN article_tags at ON at.article_id = a.id
JOIN tags t          ON t.id = at.tag_id
WHERE a.section = 'news'
ORDER BY a.title, t.title;
```

Здесь `a`, `at`, `t` — псевдонимы таблиц, чтобы не писать полные имена. Статья с двумя тегами появится в результате дважды, по строке на тег: `JOIN` размножает строки. Это нормально; ORM потом собирает их обратно в один объект со списком тегов.

Виды JOIN

Вид	Что возвращает	Пример из проекта
INNER JOIN (или просто JOIN)	только пары, где условие выполнено с обеих сторон	статьи, у которых есть теги
LEFT JOIN	все строки левой таблицы; справа NULL, если пары нет	все статьи, с тегами или без
RIGHT JOIN	зеркально LEFT; на практике переписывают в LEFT	редко
FULL JOIN	всё с обеих сторон, NULL там, где нет пары	сверка двух списков
CROSS JOIN	каждая строка с каждой	таблица умножения; почти никогда

LEFT JOIN: нули вместо пропусков

```
-- все статьи и количество тегов, включая статьи без тегов
SELECT a.slug, count(at.tag_id) AS tags_count
FROM articles a
LEFT JOIN article_tags at ON at.article_id = a.id
GROUP BY a.id, a.slug
ORDER BY tags_count DESC;
```

С INNER JOIN статьи без тегов пропали бы из результата. С LEFT JOIN они остаются, `at.tag_id` у них NULL, а `count`(колонка) NULL не считает, поэтому получаем честный ноль. Именно этот запрос стоит за нашим фильтром тегов по разделу: теги, у которых есть хотя бы одна опубликованная статья раздела.

SQL, который генерирует `ArticleService.list_tags(section)`

```
SELECT DISTINCT t.*
FROM tags t
JOIN article_tags at ON at.tag_id = t.id
JOIN articles a      ON a.id = at.article_id
WHERE a.published_at IS NOT NULL
      AND a.published_at <= now()
      AND a.section = 'news'
ORDER BY t.title;
```

8. Агрегаты и группировка

Агрегатная функция сворачивает много строк в одно значение: `count`, `sum`, `avg`, `min`, `max`. Без `GROUP BY` она считает по всей выборке; с `GROUP BY` — по каждой группе отдельно.

Агрегаты с группировкой

```
SELECT count(*) FROM articles WHERE section = 'news';           -- всего новостей

SELECT section, count(*) AS total, sum(views_count) AS views    -- по разделам
FROM articles
WHERE published_at <= now()
GROUP BY section;

SELECT t.title, count(*) AS articles                            -- популярность тегов
FROM tags t
JOIN article_tags at ON at.tag_id = t.id
GROUP BY t.id, t.title
HAVING count(*) >= 5
ORDER BY articles DESC;
```

Правило GROUP BY: в SELECT могут быть только колонки из GROUP BY и агрегаты. Нельзя написать SELECT section, title, count(*) ... GROUP BY section: у группы «news» сотня разных заголовков, база не знает, какой показать, и откажет. HAVING — это WHERE для групп: фильтрует после подсчёта, когда count(*) уже известен. WHERE работает до группировки и агрегатов не видит.

count(*) против count(колонка)

count(*) считает строки. count(cover_image) считает строки, где колонка не NULL. count(DISTINCT section) — число разных значений. В SQLAlchemy наш подсчёт общего числа статей для пагинации — это select(func.count()).select_from(Article).where(...), то есть count(*).

FILTER: несколько счётчиков одним запросом

```
SELECT
    count(*)                AS total,
    count(*) FILTER (WHERE published_at > now()) AS scheduled,
    count(*) FILTER (WHERE published_at IS NULL) AS drafts,
    min(published_at)::date AS oldest
FROM articles
WHERE section = 'news';
```

Двойное двоеточие ::date — приведение типа в PostgreSQL: из момента времени берём только день. Стандартная запись — CAST(published_at AS date).

9. Подзапросы, СТЕ и оконные функции

Подзапрос — SELECT внутри другого SELECT. Он может стоять в WHERE как список значений, в FROM как временная таблица или в SELECT как одно значение.

Три вида подзапросов

```
-- статьи, у которых есть реакции
SELECT slug FROM articles
WHERE id IN (SELECT article_id FROM article_reactions);

-- то же через EXISTS: останавливается на первой найденной строке
SELECT slug FROM articles a
WHERE EXISTS (SELECT 1 FROM article_reactions r WHERE r.article_id = a.id);

-- статьи с числом реакций выше среднего
SELECT slug, likes_count FROM articles
WHERE likes_count > (SELECT avg(likes_count) FROM articles);
```

EXISTS обычно быстрее IN на больших таблицах и читается яснее: «есть ли хоть одна такая строка». SQLAlchemy даёт для этого `exists()` и `Article.tags.any(...)`, который мы используем для фильтра по тегу.

СТЕ — именованный подзапрос

WITH имя AS (...) выносит подзапрос наверх и даёт ему имя. Запрос читается сверху вниз как программа. Именно так устроен наш ручной запрос, который расставил даты публикации задним числом:

СТЕ плюс UPDATE ... FROM

```
WITH scheduled AS (  
    SELECT id, row_number() OVER (ORDER BY published_at) AS n  
    FROM articles  
    WHERE section = 'news' AND published_at > now()  
)  
UPDATE articles AS a  
SET published_at = timestampz '2026-09-07 09:00+07' - scheduled.n * interval '1 day'  
FROM scheduled  
WHERE a.id = scheduled.id;
```

Разбор по шагам:

- СТЕ `scheduled` выбирает будущие новости и нумерует их через оконную функцию `row_number()`: 1, 2, 3... в порядке даты.
- `UPDATE ... FROM scheduled` соединяет таблицу с этим списком по `id`.
- Новая дата — 7 сентября минус `n` дней. `interval '1 day'` — тип «промежуток времени», его можно умножать на число и вычитать из даты.

Оконные функции

Оконная функция похожа на агрегат, но не сворачивает строки: каждая строка остаётся, а рядом появляется значение, вычисленное «по окну» соседних строк. `OVER (...)` задаёт окно: как сортировать и на какие группы делить.

Место статьи по просмотрам внутри своего раздела

```
SELECT  
    slug,  
    section,  
    views_count,  
    row_number() OVER (PARTITION BY section ORDER BY views_count DESC) AS place_in_section,  
    sum(views_count) OVER (PARTITION BY section) AS section_views  
FROM articles;
```

`PARTITION BY section` делит строки на группы по разделу, внутри каждой `ORDER BY views_count DESC` задаёт порядок нумерации. Через `GROUP BY` такое не выразить: там пропали бы отдельные строки.

10. INSERT, UPDATE, DELETE

Вставка одной и нескольких строк

```
INSERT INTO tags (slug, title) VALUES ('ekologiya', 'Экология');

INSERT INTO tags (slug, title)
VALUES ('sertifikaciya', 'Сертификация'),
      ('atomnaya-licenziya', 'Атомная лицензия')
RETURNING id, slug; -- вернуть созданные строки сразу
```

RETURNING — особенность PostgreSQL, которую SQLAlchemy активно использует: после `db.add(tag); await db.flush()` у объекта уже есть `tag.id` именно потому, что вставка вернула его без второго запроса.

Обновление

```
-- присвоить значение
UPDATE articles SET section = 'news' WHERE id = 42;

-- изменить относительно текущего значения: атомарно, без гонок
UPDATE articles SET views_count = views_count + 1 WHERE id = 42;

-- несколько колонок и условие
UPDATE articles
SET likes_count = likes_count - 1,
    dislikes_count = dislikes_count + 1
WHERE id = 42;
```

Почему `views_count = views_count + 1`, а не «прочитать в Python, прибавить, записать»? Потому что между чтением и записью другой запрос мог тоже прибавить единицу, и одно из увеличений потеряется. Выражение в SET вычисляется базой под блокировкой строки, поэтому сто параллельных просмотров дадут ровно сто. В сервисе статей это `update(Article).values(views_count=Article.views_count + 1)`.

Удаление

```
DELETE FROM articles WHERE id = 42; -- каскадом уйдут связи, просмотры, реакции
DELETE FROM article_reactions
WHERE article_id = 42 AND visitor_id = '651458c3-...';
```

UPDATE и DELETE без WHERE меняют или удаляют **все** строки таблицы. В psql нет кнопки «отмена». Привычка: сначала написать SELECT с тем же WHERE и посмотреть, что вернётся, затем заменить SELECT на UPDATE или DELETE.

UPSERT: вставить или обновить

ON CONFLICT: без проверки «есть ли уже»

```
INSERT INTO article_views (article_id, visitor_id)
VALUES (42, '651458c3-...')
ON CONFLICT (article_id, visitor_id) DO NOTHING;

INSERT INTO article_reactions (article_id, visitor_id, value)
VALUES (42, '651458c3-...', 1)
ON CONFLICT (article_id, visitor_id) DO UPDATE SET value = EXCLUDED.value;
```

ON CONFLICT решает ту же гонку, что и наш try/except IntegrityError при регистрации просмотра, но одним запросом. EXCLUDED — псевдотаблица со строкой, которую пытались вставить. В SQLAlchemy это `from sqlalchemy.dialects.postgresql import insert` и метод `.on_conflict_do_update(...)`; в общем sqlalchemy.insert его нет, потому что это диалектная

ВОЗМОЖНОСТЬ.

11. Транзакции

Транзакция — группа команд, которая либо выполняется целиком, либо не выполняется вовсе. Классический пример — перевод денег: списать с одного счёта и зачислить на другой должны либо оба, либо никто. Наш пример: поставить реакцию — это вставить строку в `article_reactions` и увеличить `likes_count`. Если между ними упасть, счётчик разойдётся с реальностью.

Транзакция руками

```
BEGIN;
INSERT INTO article_reactions (article_id, visitor_id, value) VALUES (42, '651458c3-...', 1);
UPDATE articles SET likes_count = likes_count + 1 WHERE id = 42;
COMMIT;      -- обе команды стали видны всем одновременно
```

```
BEGIN;
DELETE FROM articles WHERE section = 'news';
ROLLBACK;    -- передумали: ничего не удалилось
```

ACID — четыре обещания

- **Atomicity**, атомарность: всё или ничего.
- **Consistency**, согласованность: после транзакции все ограничения выполнены. Нарушили CHECK внутри — вся транзакция откатится.
- **Isolation**, изоляция: параллельные транзакции не видят незакоммиченные изменения друг друга.
- **Durability**, долговечность: после COMMIT данные переживут выключение питания.

Как это связано с сессией SQLAlchemy

Сессия SQLAlchemy — это транзакция. `async with Session() as db` открывает её при первом запросе, `await db.commit()` выполняет COMMIT, исключение или `rollback()` — ROLLBACK. Пока не было `commit`, другие соединения ваших изменений не видят. В сервисе статей `flush` отправляет INSERT в базу внутри транзакции (чтобы поймать IntegrityError или получить id), а `commit` фиксирует всё разом. Одна HTTP-заявка — одна сессия — одна транзакция: так устроена зависимость `get_session`.

Уровни изоляции и блокировки

По умолчанию PostgreSQL работает в режиме Read Committed: каждый запрос внутри транзакции видит данные, закоммиченные к моменту его начала. Для сайта этого хватает. Строгие режимы (Repeatable Read, Serializable) нужны для финансовых расчётов и стоят дороже. Блокировки база ставит сама: UPDATE строки блокирует её до конца транзакции, второй UPDATE той же строки ждёт. Поэтому держите транзакции короткими: не делайте HTTP-запросы к другим сервисам и не отправляйте письма внутри открытой сессии. У нас письмо о заявке уходит в BackgroundTasks уже после `commit`.

Гонка, которую мы ловим

Два одинаковых запроса одновременно

```
Запрос A: SELECT ... FROM article_views WHERE (article_id, visitor_id) = (42, 'x') → пусто
Запрос B: SELECT ... FROM article_views WHERE (article_id, visitor_id) = (42, 'x') → пусто
Запрос A: INSERT INTO article_views ... → ок
Запрос B: INSERT INTO article_views ... → ошибка UNIQUE (первичный ключ)
```

Проверка «есть ли уже» в коде не защищает: между SELECT и INSERT другой запрос успел вставить. Защищает первичный ключ базы: второй INSERT падает с `IntegrityError`, мы делаем `rollback` и считаем, что просмотр уже учтён. Общий принцип: уникальность гарантирует база, код лишь красиво обрабатывает отказ.

12. Эксплуатация: бэкапы, роли, часовые пояса

Резервная копия

pg_dump и восстановление

```
# снять дамп с прода в файл на сервере
cd /opt/nedra
docker compose exec -T db sh -c 'pg_dump -U "$POSTGRES_USER" -d "$POSTGRES_DB" --clean' \
  > backup-$(date +%F).sql

# восстановить из дампа
docker compose exec -T db sh -c 'psql -U "$POSTGRES_USER" -d "$POSTGRES_DB" < backup-2026-09-09.sql'
```

`pg_dump` выгружает схему и данные в виде SQL-команд: `CREATE TABLE`, `INSERT`. `--clean` добавляет `DROP` перед `CREATE`, чтобы дамп можно было накатить поверх существующей базы. Дамп — обычный текст, его можно открыть и посмотреть. Делайте его перед каждым ручным `UPDATE` в проде и перед миграциями, которые удаляют или меняют колонки.

Роли и права

Роль только для чтения, например для аналитики

```
CREATE ROLE reader LOGIN PASSWORD 'secret';
GRANT CONNECT ON DATABASE nedra TO reader;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO reader;
```

У приложения одна роль-владелец, ей можно всё. Когда появятся другие потребители базы, например BI-инструмент, заводите им отдельные роли с минимальными правами: `SELECT` и только на нужные таблицы. Пароль роли приложения меняется командой `ALTER ROLE nedra PASSWORD '...'` плюс правка `.env` и `docker compose up -d`.

Часовые пояса

Сервер в Docker живёт в UTC. Колонки `timestampz` хранят UTC и показывают в поясе сессии, поэтому в `psql` вы увидите `2026-09-07 02:00:00+00` для новосибирских `09:00`. Это одно и то же время. Чтобы `psql` показывал по-новосибирски: `SET TIME ZONE 'Asia/Novosibirsk'`; Приложение получает от `asynsprg` объект `datetime` с `tzinfo`, а фронтенд форматирует его в локальном времени браузера. Правило проекта: в базу только даты с часовым поясом.

Обслуживание

PostgreSQL не удаляет строки физически при `DELETE` и `UPDATE`, он помечает старые версии. Уборкой занимается автоматический процесс `autovacuum`, и на наших объёмах трогать его не

нужно. Логи сервера смотрят командой `docker compose logs --tail 100 db`: там видны отклонённые подключения и ошибки ограничений. Место на диске проверяют запросом `SELECT pg_size_pretty(pg_database_size('nedra'));`.

Что запомнить из части I

- База — сервер, приложение — клиент; URL описывает диалект, драйвер, хост и базу.
- Типы строгие. Время — только `timestampz`. `NULL` — «нет значения», сравнивается через `IS NULL`.
- Ограничения живут в базе и защищают данные от всех источников; имена ограничениям даём сами.
- Индексы ускоряют чтение и замедляют запись; внешние ключи индексируем руками.
- `JOIN` размножает строки, `GROUP BY` сворачивает, оконные функции делают и то и другое без потерь.
- Транзакция — всё или ничего; сессия `SQLAlchemy` — это транзакция.
- Уникальность и целостность обеспечивает база, код обрабатывает отказ.

Часть II. Основы SQLAlchemy

13. Что такое ORM и зачем он нужен

База хранит строки в таблицах. Python работает с объектами. ORM — прослойка, которая переводит одно в другое: класс становится таблицей, экземпляр — строкой, атрибут — колонкой.

```
# Без ORM
cursor.execute("SELECT id, name, email FROM requests WHERE id = %s", (5,))
row = cursor.fetchone()
name = row[1]          # надо помнить порядок колонок

# С ORM
request = await session.get(Request, 5)
name = request.name    # понятно без комментариев
```

- **Типизация.** IDE знает тип поля и подскажет опечатку до запуска.
- **Безопасность.** Параметры экранируются — SQL-инъекция через ORM практически невозможна.
- **Переносимость.** Один код работает на Postgres, MySQL, SQLite.
- **Связи.** `article.tags` вместо ручного JOIN в каждом запросе.

Цена абстракции. ORM скрывает SQL, и легко написать код, который выглядит невинно, а порождает сотню запросов. Правило: работая с ORM, всегда представляй, какой SQL получится.

14. Три кирпича: Base, Mapped, mapped_column

app/models/base.py

```
from sqlalchemy.orm import DeclarativeBase
```

```
class Base(DeclarativeBase):
    pass
```

От Base наследуются все модели. Наследование — это и есть регистрация в реестре схемы.

Mapped и mapped_column — два ответа на два вопроса

```
title: Mapped[str] = mapped_column(String(255))
#      ^^^^^^^^^^^  ^^^^^^^^^^^^^^^^^^^^^^^^^^^
#      чем будет    чем будет
#      в Python     в базе данных
```

Аннотация не декоративная — SQLAlchemy её читает и выводит из неё nullable:

```
title: Mapped[str]          # NOT NULL
cover_image: Mapped[str | None] # NULL разрешён
```

Поэтому nullable=True руками писать не нужно. Один источник правды вместо двух, которые могут разойтись.

Самая частая ошибка. Написать `content = Mapped[Text]` вместо `content: Mapped[str] = mapped_column(Text)`. Две беды сразу: знак равенства вместо двоеточия (присваивание, а не аннотация) и тип колонки внутри `Mapped[]`, где должен стоять питоновский тип. Колонки не будет, ошибки тоже.

Правило: каждая колонка — это имя : `Mapped[тип] = mapped_column(...)`. Три части, все обязательны.

15. metadata — реестр схемы

`Base.metadata` — каталог всего, что SQLAlchemy знает о базе: таблицы, колонки, индексы, внешние ключи. Объявлять не надо, его создаёт `DeclarativeBase`.

```
from app.models import Base

print(Base.metadata.tables.keys())
# dict_keys(['requests', 'articles', 'tags', 'article_tags'])
```

Реестр читает Alembic:

```
alembic/env.py

target_metadata = Base.metadata
```

Он сравнивает **реестр** с реальной схемой и генерирует разницу.

Следствие. Таблица попадает в реестр только когда Python выполнил файл с классом. Модель не импортирована в `models/__init__.py` — её нет в `metadata`, Alembic не видит разницы и создаёт **пустую миграцию**. Без ошибки и предупреждения.

```
app/models/__init__.py

from app.models.base import Base
from app.models.request import Request # noqa: F401
from app.models.article import Article, Tag # noqa: F401

__all__ = ["Base", "Request", "Article", "Tag"]
```

16. Типы колонок

Тип	Python	Когда применять
Integer	int	числа, счётчики, внешние ключи
BigInteger	int	когда int мало: суммы в копейках, telegram_id
SmallInteger	int	мелкие значения: рейтинг 1-5, номер месяца
String(n)	str	текст с известным пределом
Text	str	длинный текст: статья, комментарий
Boolean	bool	флаги
DateTime(timezone=True)	datetime	метки времени, timezone обязателен
Date	date	дата без времени: дедлайн
Time	time	время без даты: часы работы
Interval	timedelta	длительность
Numeric(10, 2)	Decimal	деньги — не Float
Float	float	измерения, где точность не критична

Тип	Python	Когда применять
Enum(PyEnum)	Enum	фиксированный набор: статус, роль
JSON / JSONB	dict, list	неструктурированные данные
ARRAY(Integer)	list[int]	массив, только Postgres
UUID	UUID	идентификатор, только Postgres
LargeBinary	bytes	бинарные данные (лучше хранить файлы вне БД)

Живой пример

agregator/models/order.py

```
class Order(Base):
    __tablename__ = "orders"

    id: Mapped[int] = mapped_column(primary_key=True)
    title: Mapped[str] = mapped_column(String(500))
    comment: Mapped[str] = mapped_column(Text, default="")
    sum_amount: Mapped[int] = mapped_column(BigInteger)
    deadline: Mapped[date] = mapped_column(Date)
    status: Mapped[OrderStatus] = mapped_column(Enum(OrderStatus), index=True)
```

Выбор осмысленный: title ограничен, comment нет; сумма в BigInteger, потому что копейки крупного заказа переполняют INTEGER; дедлайн — Date, время суток бессмысленно.

Деньги: Numeric, а не Float

```
>>> 0.1 + 0.2
0.30000000000000004          # Float — двоичная дробь, копейки теряются

price: Mapped[Decimal] = mapped_column(Numeric(10, 2))  # правильно
price: Mapped[int] = mapped_column(BigInteger)          # тоже: хранить в копейках
```

Идентификаторы — всегда строка

```
inn: Mapped[str | None] = mapped_column(String(12))  # правильно
inn: Mapped[int | None] = mapped_column(Integer)     # ошибка
```

ИНН 0123456789 в числовой колонке станет 123456789. Правило: если «число» нельзя складывать — это идентификатор, и хранить его надо текстом. Телефоны, ИНН, артикулы, номера договоров.

17. Параметры колонок

Параметр	Что делает
<code>primary_key=True</code>	первичный ключ; индекс создаётся автоматически
<code>unique=True</code>	запрет дублей на уровне БД
<code>index=True</code>	индекс для ускорения поиска
<code>nullable=False</code>	выводится из аннотации, писать не нужно
<code>default=...</code>	значение подставляет Python перед вставкой
<code>server_default=...</code>	значение подставляет база
<code>onupdate=...</code>	пересчитывается при UPDATE через ORM
<code>server_onupdate=...</code>	то же, но на стороне базы
<code>ForeignKey(...)</code>	ссылка на колонку другой таблицы
<code>comment='...'</code>	комментарий к колонке в схеме БД
<code>autoincrement=False</code>	отключить автоинкремент у целочисленного PK

`index=True` на `primary_key` не нужен. Первичный ключ индексируется автоматически. Добавив `index=True`, вы создадите второй, дублирующий индекс: лишнее место и лишняя работа при каждой вставке.

default против server_default

```
# Python ставит значение перед вставкой
created_at: Mapped[datetime] = mapped_column(default=lambda: datetime.now(UTC))
```

```
# База ставит значение сама
created_at: Mapped[datetime] = mapped_column(server_default=func.now())
```

	default	server_default
Кто вычисляет	Python	база данных
Работает при вставке мимо ORM	нет	да
Видно в схеме таблицы	нет	да
Что передавать	значение или функцию	SQL-выражение или строку

Для времени создания надёжнее база: процессов приложения может быть несколько, и часы у них разъезжаются, а база одна. Кроме того, `server_default` обязателен, когда добавляешь NOT NULL-колонку в таблицу с существующими строками — иначе миграция не пройдёт.

```
# server_default принимает SQL, а не Python-значение
is_active: Mapped[bool] = mapped_column(server_default="true")
count: Mapped[int] = mapped_column(server_default=text("0"))
created: Mapped[datetime] = mapped_column(server_default=func.now())
```

Ловушка изменяемых значений по умолчанию. Пишите `default=list`, а не `default=[]`.
Список, записанный литералом, создаётся один раз при импорте модуля и станет общим для всех строк. Передавать надо функцию.

onupdate

```
updated_at: Mapped[datetime] = mapped_column(
    DateTime(timezone=True),
    server_default=func.now(),
    onupdate=func.now(),      # пересчитается при каждом UPDATE
)
```

Работает при обновлении через ORM. Если данные меняют напрямую в SQL, нужен триггер в базе или `server_onupdate`.

18. Ограничения и индексы: `__table_args__`

Ограничения на уровне колонки задаются в `mapped_column`. Всё, что затрагивает **несколько колонок сразу**, — в `__table_args__`.

агрегатор: аккаунт

```
from sqlalchemy import CheckConstraint, Index, UniqueConstraint
```

```
class Account(Base):
    __tablename__ = "accounts"

    email: Mapped[str | None] = mapped_column(String(320))
    phone: Mapped[str | None] = mapped_column(String(20))
    role: Mapped[UserRole] = mapped_column(Enum(UserRole))

    __table_args__ = (
        CheckConstraint(
            "(email IS NOT NULL OR phone IS NOT NULL)",
            name="account_email_or_phone_required",
        ),
        UniqueConstraint("email", "role", name="uq_accounts_email_role"),
        UniqueConstraint("phone", "role", name="uq_accounts_phone_role"),
    )
```

Что здесь описано

- **CheckConstraint** — правило, которое база проверяет при каждой записи. Тут: должен быть указан хотя бы email или телефон. Логiku «или» нельзя выразить через `nullable`, поэтому она уходит в ограничение.
- **UniqueConstraint по двум колонкам** — уникальна не почта сама по себе, а пара «почта + роль». Один человек может быть и заказчиком, и экспертом с одной почтой, но не может завести два аккаунта заказчика.

Всегда давайте ограничениям имя. Без `name=` база придумает своё, и в разных окружениях оно может отличаться — Alembic начнёт генерировать миграции «на пустом месте», пытаясь переименовать ограничение туда-сюда.

Составные и частичные индексы

```
__table_args__ = (
    # составной: ускоряет WHERE status=... AND created_at > ...
    Index("ix_orders_status_created", "status", "created_at"),

    # частичный: индексируем только активные — он меньше и быстрее
    Index(
        "ix_orders_active",
        "created_at",
        postgresql_where=text("status = 'ACTIVE'"),
    ),

    # уникальный составной
    Index("uq_view_once", "article_id", "visitor_id", unique=True),
)
```

Порядок колонок в составном индексе важен: он работает для условий по первой колонке или по первой+второй, но не по одной только второй. Правило — сначала то, по чему фильтруют точным сравнением, потом диапазоны.

Когда индекс нужен

- Внешние ключи — Postgres **не создаёт** индекс на FK автоматически, а join и фильтр по нему идут постоянно.
- Колонки в WHERE и ORDER BY частых запросов.
- slug, по которому ищется каждая статья.

Индекс ускоряет чтение и замедляет запись — на каждую вставку его надо обновлять. Поэтому индексируют не всё подряд, а то, по чему реально ищут.

19. Enum в моделях

Когда у поля фиксированный набор значений — статус, роль, тип работ — его описывают перечислением. Так набор задан в одном месте, а не размазан по строковым литералам.

agregator: статус заказа

```
from enum import Enum as PyEnum
from sqlalchemy import Enum

class OrderStatus(str, PyEnum):
    ACTIVE = "ACTIVE"
    ARCHIVED = "ARCHIVED"

class Order(Base):
    status: Mapped[OrderStatus] = mapped_column(
        Enum(OrderStatus),
        index=True,
        default=OrderStatus.ACTIVE,
        server_default=OrderStatus.ACTIVE.value,
    )
```

Зачем наследоваться от str

`class OrderStatus(str, PyEnum)` — значение остаётся строкой, и его можно сравнивать, сериализовать в JSON и передавать в Pydantic без преобразований.

```
OrderStatus.ACTIVE == "ACTIVE"    # True — благодаря наследованию от str
```

Изменение Enum — это миграция. В Postgres перечисление становится отдельным типом в базе. Добавить значение можно (`ALTER TYPE ... ADD VALUE`), удалить или переименовать — гораздо сложнее. Поэтому набор значений продумывают заранее, а autogenerate изменения Enum **не отслеживает** — миграцию придётся писать руками.

Альтернатива: строка с CheckConstraint

```
status: Mapped[str] = mapped_column(String(20))

__table_args__ = (
    CheckConstraint("status IN ('ACTIVE', 'ARCHIVED')", name="ck_order_status"),
)
```

Менять набор проще (обычная миграция ограничения), но теряется типизация в Python. Выбор зависит от того, насколько часто ожидаются изменения.

20. Миксины: переиспользование колонок

`created_at` и `updated_at` нужны почти в каждой таблице. Копировать их в двадцать моделей — значит однажды где-то ошибиться.

миксин с метками времени

```
class TimestampMixin:
    created_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True), server_default=func.now()
    )
    updated_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True), server_default=func.now(), onupdate=func.now()
    )

class Article(TimestampMixin, Base):
    __tablename__ = "articles"

    id: Mapped[int] = mapped_column(primary_key=True)
    title: Mapped[str] = mapped_column(String(255))
    # created_at и updated_at пришли из миксина
```

Порядок наследования важен: миксин идёт **первым**, Base — последним.

Миксин с внешним ключом

Если в миксине нужен ForeignKey или relationship, объявлять их надо через declared_attr — иначе один и тот же объект колонки попытается принадлежать нескольким таблицам сразу.

```
from sqlalchemy.orm import declared_attr

class AuthorMixin:
    @declared_attr
    def author_id(cls) -> Mapped[int]:
        return mapped_column(ForeignKey("accounts.id", ondelete="SET NULL"))

    @declared_attr
    def author(cls) -> Mapped["Account"]:
        return relationship("Account")
```

Часть III. Связи

21. Как понять, какая связь нужна

Связей три вида, и выбрать помогает один вопрос, заданный **с обеих сторон**.

Сколько Б может быть у одного А? и Сколько А может быть у одного Б?

один — один -> 1:1
один — много -> 1:N
много — много -> M:N

Пример	Туда	Обратно	Связь
Аккаунт — профиль эксперта	один	один	1:1
Аккаунт — заказы	много	один	1:N
Заказ — бейджи	много	один	1:N
Статья — теги	много	много	M:N

Где физически лежит связь

Это главное. Связь — это **колонка с внешним ключом**, и вопрос лишь в том, в какой таблице она помещается.

Связь	Где хранится	Что писать
1:1	FK в одной из таблиц + unique	relationship(uselist=False)
1:N	FK в таблице «многих»	relationship(...)
M:N	отдельная таблица	Table + relationship(secondary=...)

Почему M:N требует третью таблицу. Куда положить колонку? В `articles` — у статьи станет один тег. В `tags` — тег будет принадлежать одной статье. Обе стороны «многие», колонке негде поместиться.

22. Один ко многим (1:N)

agregator: аккаунт и его заказы

```
class Order(Base):
    __tablename__ = "orders"

    id: Mapped[int] = mapped_column(primary_key=True)
    customer_id: Mapped[int] = mapped_column(
        ForeignKey("accounts.id", ondelete="CASCADE"), index=True
    )

    customer: Mapped["Account"] = relationship(back_populates="orders")

class Account(Base):
    __tablename__ = "accounts"

    id: Mapped[int] = mapped_column(primary_key=True)

    orders: Mapped[list["Order"]] = relationship(back_populates="customer")
```

orders

+-----+-----+			
id	customer_id	title	
+-----+-----+			
1	42	Экспертиза ПБ	
2	42	Проект котельной	
3	77	Изыскания	
+-----+-----+			

У аккаунта 42 два заказа. Одной колонки хватает.

Две стороны одной связи

`Order.customer` — один объект, `Account.orders` — список. Это одна связь с разных сторон, и связывает их `back_populates`: имя атрибута напротив.

```
account.orders.append(new_order)
new_order.customer      # Account(id=42) — поставилось само
```

Индекс на внешнем ключе

Postgres **не создаёт** индекс на FK автоматически (в отличие от первичного ключа). А запрос «все заказы аккаунта» идёт именно по этой колонке. Без индекса база читает таблицу целиком: на десяти строках незаметно, на миллионе фатально.

23. Один к одному (1:1)

agregator: аккаунт и профиль эксперта

```
class Account(Base):
    expert_profile: Mapped["Expert | None"] = relationship(
        back_populates="account",
        uselist=False,          # не список, а один объект или None
        cascade="all, delete-orphan",
        lazy="selectin",
    )

class Expert(Base):
    __tablename__ = "experts"

    account_id: Mapped[int] = mapped_column(
        ForeignKey("accounts.id", ondelete="CASCADE"), unique=True
    )
    account: Mapped["Account"] = relationship(back_populates="expert_profile")
```

unique=True на внешнем ключе — это и есть «один к одному». Без него база позволит создать два профиля для одного аккаунта, а `uselist=False` вернёт случайный. Ограничение должно стоять в схеме, а не только в коде.

Когда разделять на две таблицы

- **Разные наборы полей.** У заказа в `agregator` есть `cadastral_details`, `forensic_details`, `laboratory_details` — для каждого типа работ свой набор. Слить их в одну таблицу значит получить сотню колонок, из которых заполнены пять.
- **Необязательность.** Профиль эксперта есть не у каждого аккаунта.
- **Объём.** Редко читаемое тяжёлое поле выносят, чтобы не тащить его в каждом запросе.

24. Многие ко многим (M:N)

nedra: статьи блога и теги

```
article_tags = Table(
    "article_tags",
    Base.metadata,
    Column("article_id", ForeignKey("articles.id", ondelete="CASCADE"),
           primary_key=True),
    Column("tag_id", ForeignKey("tags.id", ondelete="CASCADE"),
           primary_key=True),
)
```

```
class Article(Base):
    __tablename__ = "articles"

    id: Mapped[int] = mapped_column(primary_key=True)
    title: Mapped[str] = mapped_column(String(255))

    tags: Mapped[list["Tag"]] = relationship(
        secondary=article_tags,
        back_populates="articles",
        lazy="selectin",
    )
```

```
class Tag(Base):
    __tablename__ = "tags"

    id: Mapped[int] = mapped_column(primary_key=True)
    title: Mapped[str] = mapped_column(String(100))

    articles: Mapped[list["Article"]] = relationship(
        secondary=article_tags,
        back_populates="tags",
    )
```

articles		tags		article_tags	
id	title	id	title	article_id	tag_id
1	Мини-НПЗ	1	Бизнес	1	1
2	Небо ТЭК	2	Безопасн.	1	2
				2	2

Зачем Base.metadata в Table

Класс попадает в реестр через наследование от Base. Table ни от чего не наследуется, поэтому реестр передают вторым аргументом. Без этого Alembic таблицу не увидит.

Почему Column, а не mapped_column

mapped_column связывает колонку с атрибутом класса и даёт типизацию. У Table класса нет — значит нет и атрибутов, нечего типизировать. Здесь голый Column.

Составной первичный ключ

primary_key=True на обеих колонках даёт ключ из пары. База сама запретит вставить (1, 1) дважды. Отдельный id здесь не нужен и вреден: с ним дубли снова станут возможны.

```
article.tags.append(tag) # SQLAlchemy сама вставит строку в article_tags
```



```
article.tags                # [Tag(...), Tag(...)] – сама сделает JOIN
```

25. Промежуточная таблица: Table или модель

Развилка, которая встречается в каждом проекте. Ответ зависит от одного: **есть ли у самой связи собственные данные**.

Вариант А: Table — связь это просто факт

«У статьи есть тег». Больше сказать нечего — значит Table.

Вариант В: модель — у связи есть свойства

«Эксперт аттестован по специализации **с такого-то года**, документ **такой-то**». Эти поля не про эксперта и не про специализацию, а именно про их связь. Такую модель называют association object.

```
class ExpertSpecialization(Base):
    __tablename__ = "expert_specializations"

    expert_id: Mapped[int] = mapped_column(
        ForeignKey("experts.id", ondelete="CASCADE"), primary_key=True
    )
    specialization_id: Mapped[int] = mapped_column(
        ForeignKey("specializations.id", ondelete="CASCADE"), primary_key=True
    )

    # ради этого и понадобился класс
    certified_since: Mapped[date] = mapped_column(Date)
    certificate_number: Mapped[str] = mapped_column(String(100))

    expert: Mapped["Expert"] = relationship(back_populates="specializations")
    specialization: Mapped["Specialization"] = relationship(
        back_populates="experts"
    )
```

Разница в использовании. С Table работаешь со списком целевых объектов: `article.tags` — список Tag. С association object — список **связей**, и до специализации надо идти через `.specialization`. Менее удобно, поэтому не превращайте связь в модель без нужды.

Вопрос	Table	Модель
У связи есть свои поля?	нет	да
Нужна дата создания связи?	нет	да
Связь надо менять, а не только добавлять и удалять?	нет	да
Пример	статья — тег	эксперт — специализация

Формулировка для собеседования. «Промежуточная таблица без своих полей описывается через Table и подключается параметром `secondary`. Если у связи появляются собственные атрибуты, она становится association object — обычной моделью с двумя внешними ключами в составном первичном ключе».

26. Параметры relationship

Параметр	Зачем
back_populates	имя атрибута на другой стороне; держит стороны в согласии
secondary	промежуточная таблица для M:N
uselist=False	связь 1:1 — один объект вместо списка
cascade	что делать со связанными объектами при операциях с родителем
passive_deletes=True	доверить каскадное удаление базе
lazy	стратегия загрузки
foreign_keys	какой FK использовать, если их несколько
order_by	порядок элементов в списке
primaryjoin	своё условие соединения, когда FK недостаточно
viewonly=True	связь только для чтения, изменения не пишутся
single_parent=True	объект может принадлежать только одному родителю

foreign_keys — когда связей несколько

две связи к одной таблице

```
class Order(Base):
    customer_id: Mapped[int] = mapped_column(ForeignKey("accounts.id"))
    assigned_expert_id: Mapped[int | None] = mapped_column(
        ForeignKey("accounts.id")
    )

    customer: Mapped["Account"] = relationship(
        foreign_keys=[customer_id], back_populates="orders"
    )
    assigned_expert: Mapped["Account | None"] = relationship(
        foreign_keys=[assigned_expert_id], back_populates="assigned_orders"
    )
```

Без `foreign_keys` приложение не запустится: SQLAlchemy не сможет определить условие соединения.

order_by и viewonly

```
comments: Mapped[list["Comment"]] = relationship(
    order_by="Comment.created_at.desc()", # свежие первыми
)

last_message: Mapped["Message | None"] = relationship(
    primaryjoin="and_(Chat.id == Message.chat_id, Message.is_last == True)",
    viewonly=True, # вычисляемая связь, писать в неё нельзя
)
```

27. Загрузка данных: lazy, N+1 и async

Самая важная глава. Здесь живут и главная ошибка производительности, и главная ошибка async.

Проблема N+1

```
articles = (await session.execute(select(Article).limit(12))).scalars().all()

for article in articles:
    print(article.tags)      # каждое обращение – отдельный запрос

SELECT * FROM articles LIMIT 12;                -- 1 запрос
SELECT * FROM tags JOIN article_tags ... WHERE article_id=1; -- +1
SELECT * FROM tags JOIN article_tags ... WHERE article_id=2; -- +1
...                                              -- ещё 10
                                              итого 13 запросов
```

Один запрос за списком плюс по одному на каждый элемент. На списке из 1000 записей — 1001 запрос.

Стратегии загрузки

Задаются параметром lazy= в relationship.

Значение lazy	Как работает	Когда
lazy="select" (по умолчанию)	отдельный запрос в момент обращения	почти никогда в async
lazy="selectin"	доп. запрос WHERE id IN (...)	списки — лучший выбор
lazy="joined"	JOIN в основном запросе	1:1 и «многие к одному»
lazy="subquery"	подзапрос	устарел, вместо него selectin
lazy="raise"	исключение при ленивой загрузке	чтобы ловить N+1 на тестах
lazy="noload"	всегда пустой список	когда связь точно не нужна

Не путайте с функцией select(). Это разные вещи с одинаковым именем. select(Article) — функция построения запроса, вы пишете её сами и выполняете через await session.execute(...); она нужна всегда и в async работает прекрасно. lazy="select" — стратегия, при которой SQLAlchemy пытается сходить в базу **сама**, в момент чтения article.tags, без await. Вот это в async и невозможно. Правило простое: явный запрос с await работает, неявный без него — падает.

Два способа задать стратегию

```
# 1. В модели – всегда так грузить
tags: Mapped[list["Tag"]] = relationship(secondary=article_tags,
                                         lazy="selectin")

# 2. В конкретном запросе – только здесь
```

```

from sqlalchemy.orm import selectinload, joinedload

stmt = select(Article).options(selectinload(Article.tags)).limit(12)

# вложенные связи
stmt = select(Order).options(
    selectinload(Order.responses).selectinload(OrderResponse.expert)
)

```

Первый способ удобнее, когда связь нужна почти всегда (теги статьи, бейджи заказа). Второй — когда она нужна редко.

В async ленивая загрузка не работает вообще. Обращение к незагруженной связи выбросит `MissingGreenlet`. Ленивая подгрузка делает синхронный запрос в момент чтения атрибута, а в асинхронном коде синхронных запросов быть не может. Поэтому стратегию нужно задавать **всегда** — в модели или в запросе. Это не оптимизация, а обязательное условие.

Почему `selectin`, а не `joined`

`joined` делает один запрос с `JOIN` — идеально для «многие к одному». Но для списка «один ко многим» `JOIN` размножает строки: 12 статей по 3 тега дадут 36 строк, где данные статей продублированы трижды. `selectin` делает второй запрос с `WHERE id IN (...)` — данных по сети меньше.

Как увидеть свои запросы

```
engine = create_async_engine(url, echo=True) # печатает весь SQL в консоль
```

28. Удаление: cascade и ondelete

Похожие вещи настраиваются в двух разных местах и работают на разных уровнях.

	cascade	ondelete
Где пишется	в relationship	в ForeignKey
Кто исполняет	SQLAlchemy (Python)	база данных
Как работает	выгружает детей, удаляет по одному	удаляет одним запросом
Работает мимо ORM?	нет	да

agregator: правильная связка

```
class Order(Base):
    badges: Mapped[list["OrderBadge"]] = relationship(
        back_populates="order",
        cascade="all, delete-orphan",    # уровень ORM
        passive_deletes=True,            # доверить работу базе
    )

class OrderBadge(Base):
    order_id: Mapped[int] = mapped_column(
        ForeignKey("orders.id", ondelete="CASCADE")    # уровень БД
    )
```

- **cascade="all, delete-orphan"** — удаляя заказ через ORM, удалить и бейджи. delete-orphan добавляет: бейдж, выброшенный из списка, тоже удаляется — он потерял родителя.
- **ondelete="CASCADE"** — то же правило в самой базе. Сработает даже при удалении напрямую через SQL.
- **passive_deletes=True** — не выгружать детей в память ради удаления. Без него SQLAlchemy сначала прочитает все бейджи, потом удалит по одному.

Значения cascade

Значение	Что делает
save-update	сохранять связанные объекты вместе с родителем (по умолчанию)
merge	объединять связанные при merge()
delete	удалять детей при удалении родителя
delete-orphan	удалять ребёнка, отвязанного от родителя
all	сокращение для save-update, merge, refresh-expire, expunge, delete

ondelete: варианты

Значение	Что происходит при удалении родителя
CASCADE	дети удаляются: бейджи заказа, связи с тегами
SET NULL	ссылка обнуляется: у заказа снимается назначенный эксперт
RESTRICT	удаление запрещено, пока есть дети
NO ACTION	по умолчанию: ошибка внешнего ключа

В `agregator` выбор осмысленный: у заказа `customer_id` — `CASCADE` (нет заказчика — нет заказа), а `assigned_expert_id` — `SET NULL` (эксперт ушёл, заказ остался и его можно переназначить).

Часть IV. Запросы

29. select: основы

```
from sqlalchemy import select

# по первичному ключу — самый короткий путь
article = await session.get(Article, 5)

# по условию
stmt = select(Article).where(Article.slug == "mini-npz")
article = (await session.execute(stmt)).scalar_one_or_none()

# список с фильтром, сортировкой и пагинацией
stmt = (
    select(Article)
    .where(Article.published_at.is_not(None))
    .order_by(Article.published_at.desc())
    .limit(12)
    .offset(0)
)
articles = list((await session.execute(stmt)).scalars().all())
```

Методы результата

Метод	Возвращает
<code>scalar_one_or_none()</code>	объект или <code>None</code> ; ошибка, если найдено больше одного
<code>scalar_one()</code>	ровно один; ошибка и когда ничего не найдено
<code>scalars().all()</code>	список объектов
<code>scalars().first()</code>	первый или <code>None</code>
<code>session.scalar(stmt)</code>	сразу одно значение — удобно для <code>COUNT</code>
<code>all()</code>	список кортежей (когда выбираешь несколько колонок)

Запрос строится по частям

```
stmt = select(Article)

if tag_slug:
    stmt = stmt.join(Article.tags).where(Tag.slug == tag_slug)

if only_published:
    stmt = stmt.where(Article.published_at.is_not(None))

stmt = stmt.order_by(Article.published_at.desc()).limit(limit)
```

Каждый вызов возвращает **новый** объект запроса, а не меняет старый. Поэтому результат обязательно присваивать обратно — `stmt.where(...)` без `stmt =` не сделает ничего.

Выбор отдельных колонок

```
# только нужные поля — меньше данных по сети
```



```
stmt = select(Article.id, Article.title, Article.views_count)
rows = (await session.execute(stmt)).all()

for row in rows:
    print(row.title, row.views_count)  # доступ по имени
```

30. Операторы и условия

Python	SQL	Пример
<code>==</code>	<code>=</code>	<code>Article.slug == 'npz'</code>
<code>!=</code>	<code><></code>	<code>Order.status != 'ARCHIVED'</code>
<code>> >= < <=</code>	<code>> >= < <=</code>	<code>Article.views_count >= 100</code>
<code>.in_(...)</code>	<code>IN</code>	<code>Article.id.in_([1, 2, 3])</code>
<code>.not_in(...)</code>	<code>NOT IN</code>	<code>Tag.slug.not_in(['spam'])</code>
<code>.is_(None)</code>	<code>IS NULL</code>	<code>Article.published_at.is_(None)</code>
<code>.is_not(None)</code>	<code>IS NOT NULL</code>	<code>Article.cover_image.is_not(None)</code>
<code>.like('%x%')</code>	<code>LIKE</code>	<code>Article.title.like('%НПЗ%')</code>
<code>.ilike('%x%')</code>	<code>ILIKE</code>	<code>Article.title.ilike('%нпз%')</code>
<code>.between(a, b)</code>	<code>BETWEEN</code>	<code>Order.deadline.between(d1, d2)</code>
<code>.startswith('x')</code>	<code>LIKE 'x%'</code>	<code>Tag.slug.startswith('prom')</code>
<code>.contains(x)</code>	для связей	<code>Article.tags.contains(tag)</code>
<code>.any()</code>	<code>EXISTS</code>	<code>Article.tags.any(Tag.slug == 'npz')</code>

Логические операторы

```
from sqlalchemy import and_, or_, not_

# И — просто перечисление через запятую
select(Article).where(
    Article.published_at.is_not(None),
    Article.views_count > 100,
)

# то же явно
select(Article).where(and_(...))

# ИЛИ
select(Tag).where(or_(Tag.slug == "npz", Tag.slug == "bpla"))

# отрицание
select(Article).where(not_(Article.slug.startswith("draft")))
```

Не используйте Python-операторы `and` / `or` / `not`. Они работают с истинностью объекта, а не строят SQL. Нужны именно `and_`, `or_`, `not_` — либо перечисление через запятую для «И».

Фильтр по связи

```
# статьи, у которых есть тег с таким slug — без JOIN, через EXISTS
stmt = select(Article).where(Article.tags.any(Tag.slug == "promyshlennost"))
```

```
# у заказа есть хотя бы один отклик
stmt = select(Order).where(Order.responses.any())
```

```
# все отклики удовлетворяют условию
stmt = select(Order).where(~Order.responses.any(OrderResponse.price > 100000))
```

`.any()` разворачивается в `EXISTS` и **не даёт дублей**, в отличие от `JOIN` — об этом в разделе про соединения.

31. Функции SQL: func

`func` — это мост к любым функциям базы. SQLAlchemy не держит их список: `func.что_угодно()` превращается в вызов `что_угодно()` в SQL. Поэтому доступны и стандартные функции, и специфичные для Postgres.

```
from sqlalchemy import func
```

```
func.count()           -> count()
func.now()             -> now()
func.coalesce(a, b)    -> coalesce(a, b)
func.jsonb_agg(x)      -> jsonb_agg(x)    — специфично для Postgres
```

Агрегатные функции

Функция	Что делает	Пример
<code>func.count()</code>	количество строк	<code>select(func.count()).select_from(Article)</code>
<code>func.count(col)</code>	количество не-NULL значений	<code>func.count(Article.cover_image)</code>
<code>func.count(distinct(col))</code>	количество уникальных	<code>func.count(distinct(Order.customer_id))</code>
<code>func.sum(col)</code>	сумма	<code>func.sum(Order.sum_amount)</code>
<code>func.avg(col)</code>	среднее	<code>func.avg(Article.views_count)</code>
<code>func.min / func.max</code>	минимум и максимум	<code>func.max(Article.published_at)</code>

Работа с датой и временем

Функция	Что делает
<code>func.now()</code>	текущее время базы; для <code>server_default</code>
<code>func.current_date()</code>	сегодняшняя дата
<code>func.date_trunc('month', col)</code>	округление вниз: начало месяца, дня, часа
<code>func.extract('year', col)</code>	вытащить часть даты
<code>func.age(col)</code>	разница со текущим моментом (Postgres)
<code>func.to_char(col, 'DD.MM.YYYY')</code>	форматирование в строку (Postgres)

pedra: сколько заявок приходит каждый месяц

```
# статистика заявок по месяцам
stmt = (
    select(
        func.date_trunc("month", Request.created_at).label("month"),
        func.count().label("total"),
    )
    .group_by("month")
    .order_by("month")
)
```

Работа со строками

Функция	Что делает
<code>func.lower / func.upper</code>	регистр
<code>func.length(col)</code>	длина строки
<code>func.trim(col)</code>	убрать пробелы по краям
<code>func.concat(a, b)</code>	склеить (или оператор <code>+</code>)
<code>func.replace(col, a, b)</code>	заменить подстроку
<code>func.substr(col, 1, 100)</code>	часть строки

coalesce — подстановка значения вместо NULL

```
# если описания нет, показать первые 200 символов текста
stmt = select(
    Article.title,
    func.coalesce(Article.description, func.substr(Article.content, 1, 200)),
)

# сумма по пустой выборке даёт NULL — превращаем в 0
total = await session.scalar(
    select(func.coalesce(func.sum(Order.sum_amount), 0))
)
```

Про sum и пустую выборку. COUNT по нулю строк вернёт 0, а SUM — NULL. В Python это станет None, и арифметика с ним упадёт. Оборачивайте суммы в coalesce.

Функции в выражениях колонок

```
# инкремент средствами базы — атомарно
update(Article).values(views_count=Article.views_count + 1)

# нормализация при поиске
select(Account).where(func.lower(Account.email) == email.lower())
```

Обратите внимание: `Article.views_count + 1` — это **SQL-выражение**, а не вычисление в Python. Значение читает и увеличивает сама база.

label — имя для вычисляемой колонки

```
stmt = select(
    Tag.title,
    func.count(Article.id).label("articles_count"),
).join(Tag.articles).group_by(Tag.id)

for row in (await session.execute(stmt)).all():
    print(row.title, row.articles_count)  # обращение по метке
```

32. Агрегация и группировка

GROUP BY схлопывает строки в группы, а агрегатные функции считают по каждой группе.

nedra: популярность направлений

```
# сколько заявок по каждому направлению
stmt = (
    select(
        Request.direction,
        func.count().label("total"),
    )
    .group_by(Request.direction)
    .order_by(func.count().desc())
)

rows = (await session.execute(stmt)).all()
# [(2, 47), (0, 31), (5, 12), ...]
```

HAVING — фильтр по результату агрегации

```
# теги, у которых больше 5 статей
stmt = (
    select(Tag.title, func.count(Article.id).label("cnt"))
    .join(Tag.articles)
    .group_by(Tag.id)
    .having(func.count(Article.id) > 5)
)
```

WHERE или HAVING. WHERE отсеивает строки **до** группировки, HAVING — группы **после**. Условие по обычной колонке — в WHERE (так быстрее, строк меньше), условие по агрегату — только в HAVING.

```
stmt = (
    select(Tag.title, func.count(Article.id))
    .join(Tag.articles)
    .where(Article.published_at.is_not(None)) # до группировки
    .group_by(Tag.id)
    .having(func.count(Article.id) > 5)      # после
)
```

Несколько агрегатов сразу

```
stmt = select(
    func.count().label("total"),
    func.sum(Article.views_count).label("views"),
    func.avg(Article.likes_count).label("avg_likes"),
    func.max(Article.published_at).label("last_published"),
)

stats = (await session.execute(stmt)).one()
print(stats.total, stats.views, stats.avg_likes)
```

Оконные функции

Считают по группе, но **не схлопывают** строки — каждая строка остаётся и получает своё значение.

```
# место статьи по просмотрам внутри своего тега
stmt = select(
```

```
Article.title,  
func.row_number().over(  
    partition_by=Tag.id,  
    order_by=Article.views_count.desc(),  
).label("rank"),  
).join(Article.tags)
```

Полезные оконные функции: `row_number()` — порядковый номер, `rank()` — место с пропусками при равенстве, `lag()` / `lead()` — значение из соседней строки, `sum().over()` — нарастающий итог.

33. JOIN: соединение таблиц

```
# по связи — SQLAlchemy сама знает условие
stmt = select(Article).join(Article.tags).where(Tag.slug == "npz")

# явно по таблице и условию
stmt = select(Order).join(Account, Order.customer_id == Account.id)

# LEFT JOIN — строки левой таблицы остаются, даже если справа ничего нет
stmt = select(Article).outerjoin(Article.tags)
```

Главная ловушка JOIN — дубли. Если у статьи три тега, JOIN вернёт её трижды. Для фильтра «есть такой тег» это лишнее. Решения: `.distinct()`, либо — лучше — `.where(Article.tags.any(...))`, которое разворачивается в EXISTS и дублей не даёт.

```
# даёт дубли, если тегов несколько
select(Article).join(Article.tags).where(Tag.slug.in_(["a", "b"]))

# дублей нет
select(Article).where(Article.tags.any(Tag.slug.in_(["a", "b"])))
```

JOIN не загружает связь

```
stmt = (
    select(Article)
    .join(Article.tags)                # для фильтрации
    .where(Tag.slug == "promyshlennost")
    .options(selectinload(Article.tags)) # для загрузки
)
```

Это разные задачи. `join` позволяет написать условие по связанной таблице, но **не кладёт** теги в объекты. Без `options` обращение к `article.tags` вызовет ленивую загрузку — и падение в `asyns`.

34. Подзапросы, CTE и exists

exists — есть ли хоть одна строка

```
from sqlalchemy import exists

# аккаунты, у которых есть хотя бы один заказ
stmt = select(Account).where(
    exists().where(Order.customer_id == Account.id)
)

# проверка одной записи — дешевле, чем считать все
has_view = await session.scalar(
    select(exists().where(
        ArticleView.article_id == article_id,
        ArticleView.visitor_id == visitor_id,
    ))
)
```

EXISTS останавливается на первой найденной строке, поэтому он всегда быстрее, чем `COUNT(*) > 0`.

Подзапрос

```
# средние просмотры
avg_views = select(func.avg(Article.views_count)).scalar_subquery()

# статьи выше среднего
stmt = select(Article).where(Article.views_count > avg_views)
```

СТЕ — именованный подзапрос

```
# топ-10 статей по просмотрам, затем их теги
top = (
    select(Article.id, Article.title, Article.views_count)
    .order_by(Article.views_count.desc())
    .limit(10)
    .cte("top_articles")
)

stmt = (
    select(top.c.title, Tag.title)
    .join(article_tags, article_tags.c.article_id == top.c.id)
    .join(Tag, Tag.id == article_tags.c.tag_id)
)
```

СТЕ читается сверху вниз и разбивает сложный запрос на понятные шаги. Обращение к колонкам — через `.c`: `top.c.title`.

35. Изменение данных

Через объекты

```
# вставка
article = Article(slug="npz", title="Мини-НПЗ", content="...")
session.add(article)
await session.commit()

# обновление — просто присваиваем
article.title = "Новый заголовок"
await session.commit()          # SQLAlchemy сама соберёт UPDATE

# удаление
await session.delete(article)
await session.commit()
```

Отдельного «сохранить» нет: SQLAlchemy отслеживает изменения загруженных объектов и на commit собирает нужный SQL.

Массовые операции

Когда объекты не нужны, работать напрямую запросом быстрее — не надо загружать строки в память.

```
from sqlalchemy import delete, insert, update

# атомарный инкремент
await session.execute(
    update(Article)
    .where(Article.id == article_id)
    .values(views_count=Article.views_count + 1)
)

# массовое обновление
await session.execute(
    update(Order)
    .where(Order.deadline < date.today())
    .values(status=OrderStatus.ARCHIVED)
)

# удаление по условию
await session.execute(delete(ArticleView).where(ArticleView.created_at < cutoff))

# вставка нескольких строк одним запросом
await session.execute(insert(Tag), [
    {"slug": "npz", "title": "Мини-НПЗ"},
    {"slug": "bpla", "title": "БПЛА"},
])
```

Массовые операции проходят мимо ORM. Объекты, уже загруженные в сессию, об изменении не узнают, а cascade уровня ORM не сработает — каскадное удаление отработает только через ondelete в базе. Это плата за скорость.

UPSERT — вставить или обновить

nedra: уникальный просмотр статьи

```
from sqlalchemy.dialects.postgresql import insert

# просмотр: если уже был, ничего не делаем
stmt = (
    insert(ArticleView)
    .values(article_id=article_id, visitor_id=visitor_id)
    .on_conflict_do_nothing(index_elements=["article_id", "visitor_id"])
)
result = await session.execute(stmt)

# сработала ли вставка – от этого зависит, увеличивать ли счётчик
if result.rowcount:
    await session.execute(
        update(Article)
        .where(Article.id == article_id)
        .values(views_count=Article.views_count + 1)
    )
```

nedra: смена реакции

```
# реакция: поставил лайк, потом передумал и жмёт дизлайк
stmt = (
    insert(ArticleReaction)
    .values(article_id=article_id, visitor_id=visitor_id, value=1)
    .on_conflict_do_update(
        index_elements=["article_id", "visitor_id"],
        set_={"value": 1},
    )
)
```

Почему UPSERT, а не «проверить и вставить». Между проверкой и вставкой успевают вклиниться второй запрос, и оба вставят строку. Уникальность гарантирует только ограничение UNIQUE в схеме плюс ON CONFLICT. Это не оптимизация, а корректность.

returning — получить данные изменённых строк

```
stmt = (
    update(Article)
    .where(Article.id == article_id)
    .values(views_count=Article.views_count + 1)
    .returning(Article.views_count)
)
new_count = await session.scalar(stmt) # актуальное значение без второго запроса
```

36. Сессия: flush, commit, refresh

Сессия — «черновик» изменений. Она копит их в памяти и решает, когда отправить в базу.

Метод	Что делает
<code>add(obj)</code>	положить объект в сессию; SQL ещё не выполнялся
<code>flush()</code>	отправить SQL, но транзакцию не завершать; появляются <code>id</code>
<code>commit()</code>	зафиксировать транзакцию — данные видны всем
<code>rollback()</code>	откатить всё с начала транзакции
<code>refresh(obj)</code>	перечитать объект из базы
<code>expunge(obj)</code>	убрать объект из сессии, не удаляя из базы
<code>merge(obj)</code>	вливать состояние отсоединённого объекта в сессию

nedra: создание заявки

```
async def create_request(self, data: RequestInSchema) -> Request:
    request = Request(name=data.name, email=data.email, ...)

    self.db.add(request)          # в сессии, в базе ещё нет
    await self.db.commit()        # INSERT выполнен, транзакция закрыта
    await self.db.refresh(request) # подтянули id и created_at
    return request
```

Зачем refresh

`id` генерирует база (автоинкремент), `created_at` тоже (`server_default`). До вставки этих значений в Python-объекте нет.

Зачем flush

```
article = Article(title="...", slug="...")
self.db.add(article)
await self.db.flush()          # id появился, транзакция ещё открыта

for tag in tags:
    self.db.add(ArticleTag(article_id=article.id, tag_id=tag.id))

await self.db.commit()         # всё вместе одной транзакцией
```

Ключевая идея транзакции: либо все изменения применятся, либо ни одного. Если между `flush` и `commit` произойдёт ошибка, статья тоже не сохранится. Именно поэтому связанные записи создают в одной транзакции, а не отдельными коммитами.

`expire_on_commit=False`

```
SessionLocal = async_sessionmaker(engine, expire_on_commit=False)
```

По умолчанию после `commit` SQLAlchemy помечает атрибуты устаревшими, и следующее обращение к ним лезет в базу. В `async` это даст `MissingGreenlet` при сборке ответа. Поэтому в `async`-проектах флаг ставят почти всегда.

Одна сессия на запрос

nedra/app/database.py

```
async def get_session() -> AsyncGenerator[AsyncSession, None]:  
    async with SessionLocal() as session:  
        yield session
```

Сессию открывает зависимость, а `commit` делает сервис — только он знает, где заканчивается бизнес-операция. При исключении внутри `async with` сессия закроется без `commit`, то есть транзакция откатится сама.

Часть V. Движок, сессии и FastAPI

Модели описывают таблицы, запросы описывают, что достать. Осталось понять, как всё это физически доезжает до PostgreSQL и обратно: кто открывает соединение, кто держит транзакцию, почему у нас всё асинхронное и где именно SQLAlchemy встречается с FastAPI.

37. Engine: соединения, пул и async

Engine — точка входа SQLAlchemy в базу. Он хранит URL, знает диалект и драйвер и управляет пулом соединений. Создаётся один раз на всё приложение.

backend/app/database.py

```
from sqlalchemy.ext.asyncio import AsyncSession, async_sessionmaker, create_async_engine

from app.config import get_settings

settings = get_settings()

engine = create_async_engine(settings.database_url, echo=settings.debug)

SessionLocal = async_sessionmaker(engine, class_=AsyncSession, expire_on_commit=False)

async def get_session():
    """Сессия на один HTTP-запрос: открыть, отдать роутеру, закрыть."""

    async with SessionLocal() as session:
        yield session
```

Что откуда импортируем

Имя	Модуль	Зачем
create_async_engine	sqlalchemy.ext.asyncio	создать движок под асинхронный драйвер asyncpg
AsyncSession	sqlalchemy.ext.asyncio	класс сессии с await у execute, commit, refresh
async_sessionmaker	sqlalchemy.ext.asyncio	фабрика сессий с общими настройками
create_engine, Session, sessionmaker	sqlalchemy, sqlalchemy.orm	то же для синхронного кода; в проекте не используются

Пул соединений

Открыть TCP-соединение с PostgreSQL и пройти аутентификацию — дорого, десятки миллисекунд. Поэтому движок держит пул: несколько открытых соединений, которые сессии берут на время запроса и возвращают. По умолчанию пять постоянных плюс до десяти временных. Для сайта с десятками запросов в секунду этого хватает; если увидите ошибку `QueuePool limit ... reached`, значит, соединения не возвращаются: где-то сессия не закрывается

или транзакция висит.

Настройки пула, которые имеет смысл знать

```
engine = create_async_engine(
    settings.database_url,
    pool_size=5,          # постоянных соединений
    max_overflow=10,      # временных сверх pool_size под пиковую нагрузку
    pool_pre_ping=True,   # перед выдачей проверить, живо ли соединение
    pool_recycle=1800,    # пересоздавать соединения старше 30 минут
    echo=False,          # печатать весь SQL в лог: включать только при отладке
)
```

`pool_pre_ping` спасает от ситуации «база перезапустилась, а в пуле лежат мёртвые соединения»: перед выдачей движок отправляет `SELECT 1` и при ошибке открывает новое. После `docker compose restart db` без этого флага первые запросы бэкенда упадут.

Синхронно или асинхронно

FastAPI построен на `asyncio`: один процесс обслуживает много запросов, переключаясь между ними в точках `await`. Если внутри такого процесса вызвать блокирующий драйвер `psycopg2`, весь процесс встанет и будет ждать ответа базы, другие запросы замрут. Асинхронный драйвер `asyncpg` отдаёт управление на время ожидания. Поэтому в проекте всё асинхронное: `create_async_engine`, `AsyncSession`, `await db.execute(...)`, и роутеры объявлены как `async def`.

Цена асинхронности — одна: ленивая подгрузка не работает. Когда вы обращаетесь к `article.tags`, а теги ещё не загружены, синхронный `SQLAlchemy` тихо сделает запрос. Асинхронный не может: обращение к атрибуту — не `await`, и выполнить запрос негде. Отсюда ошибка `MissingGreenlet` и наши правила: `lazy='selectin'` у связей, которые нужны всегда, и явная перечитка объекта после `commit`. Подробно — в главе 27, здесь важно понять причину.

echo и логи SQL

`echo=True` печатает каждый SQL-запрос с параметрами. Это лучший способ увидеть, что на самом деле генерирует ORM, поймать N+1 и лишние запросы. Включайте локально через `DEBUG=true`, на проде держите выключенным: лог раздувается мгновенно.

38. Сессия: жизненный цикл, identity map, expire_on_commit

Сессия — рабочее пространство одной единицы работы. Она берёт соединение из пула, открывает транзакцию, следит за объектами, которые вы через неё загрузили или добавили, и в нужный момент отправляет изменения в базу.

Жизненный цикл в одном запросе

Что происходит на каждом шаге

```
async with SessionLocal() as db:
    article = await db.get(Article, 42)
    article.title = "Новый заголовок"
    tag = Tag(slug="eko", title="Эко")
    db.add(tag)
    await db.flush()
    print(tag.id)
    await db.commit()
```

1. сессия создана, соединения пока нет
2. первый запрос: взяли соединение, BEGIN
3. объект «грязный», в базу ещё ничего не ушло
4. объект «ожидаящий», INSERT ещё не отправлен
5. UPDATE и INSERT ушли в базу, транзакция открыта
6. id уже есть благодаря RETURNING
7. COMMIT: изменения видны всем
8. выход из with: соединение вернулось в пул

Unit of Work: сессия копит изменения

Вы меняете атрибуты объектов как обычные поля Python, а сессия запоминает, что изменилось. При flush она сама сортирует операции в правильном порядке: сначала вставить родителя, потом детей с внешним ключом на него, обновления сгруппировать. Это паттерн Unit of Work. Поэтому в сервисе статей нет ни одного явного UPDATE для полей: `article.title = changes['title']` и потом один `commit`.

flush против commit

- `flush` отправляет накопленные INSERT/UPDATE/DELETE в базу, но транзакцию не закрывает. Нужен, когда хотите получить `id` до `commit` или поймать `IntegrityError` в конкретном месте, как мы делаем при регистрации просмотра.
- `commit` делает `flush` и затем COMMIT. После него транзакция закрыта, следующая операция откроет новую.
- `rollback` откатывает транзакцию и сбрасывает состояние объектов к тому, что было. После `IntegrityError` сессия непригодна, пока не сделать `rollback`.

Identity map: один объект на одну строку

Сессия хранит словарь «первичный ключ → объект». Если вы дважды запросите статью с `id` 42 в одной сессии, получите один и тот же объект Python, а не две копии. Это удобно: изменения не расходятся. Но это же — причина ловушки, которую мы поймали.

Почему db.get после commit не помог

```
article = await db.get(Article, article_id)
article.title = "..."
await db.commit()
again = await db.get(Article, article_id)
print(again.updated_at)
```

тот же объект из identity map, запроса в базу нет
MissingGreenlet: атрибут протух после commit

После `commit` SQLAlchemy помечает колонки с `onupdate` и `server_default` как устаревшие: их значения вычислила база, Python их не знает. Обращение к такому атрибуту требует запроса, а в `async` это невозможно. `db.get` вернул тот же объект из `identity map`, не ходя в базу. Решение — заставить сессию перечитать строку и перезаписать объект:

populate_existing: обновить объект в identity map данными из базы

```
async def reload(self, article_id: int) -> Article:
    """Перечитать статью из базы после записи."""

    stmt = (
        select(Article)
        .where(Article.id == article_id)
        .execution_options(populate_existing=True)
    )

    result = await self.db.execute(stmt)

    return result.scalar_one()
```

Альтернатива — `await db.refresh(article)`: перечитывает один объект. Мы выбрали `populate_existing`, потому что он заодно подгружает теги через `selectin`, а `refresh` связи по умолчанию не трогает.

expire_on_commit

По умолчанию после `commit` сессия помечает **все** атрибуты всех объектов устаревшими, и любое обращение к ним требует запроса. В синхронном коде это просто лишние запросы, в `async` — ошибка. Поэтому у нас `expire_on_commit=False`: объекты после `commit` остаются читаемыми. Обратная сторона — колонки, вычисленные базой, надо перечитывать явно, что мы и делаем в `reload`.

Сессия на запрос: зависимость FastAPI

Цепочка зависимостей: `get_session` → `сервис` → `роутер`

```
from fastapi import Depends
from sqlalchemy.ext.asyncio import AsyncSession

from app.database import get_session
from app.services.articles import ArticleService

def get_article_service(
    session: AsyncSession = Depends(get_session),
) -> ArticleService:
    """Сервис статей с сессией текущего запроса."""

    return ArticleService(session)

@router.get("/articles/{slug}")
async def get_article(
    slug: str,
    service: ArticleService = Depends(get_article_service),
) -> ArticleSchema:
    article = await service.get_published(slug)

    return ArticleSchema.model_validate(article)
```

FastAPI вызывает `get_session` в начале запроса, доходит до `yield` и отдаёт сессию дальше по цепочке. Когда ответ отправлен, выполнение продолжается после `yield`: блок `async with` закрывается, соединение возвращается в пул. Если внутри было исключение, сессия откатится. Так каждая заявка получает свою транзакцию, а роутер не знает ничего о движке.

Почему сервис, а не запросы в роутере

Роутер отвечает за HTTP: разобрать параметры, вернуть код ответа, превратить доменную ошибку в 404 или 422. Сервис отвечает за данные: запросы, транзакции, правила («черновик считается найденным»). Разделение даёт две вещи: сервис можно вызвать из скрипта или теста без HTTP, а роутер читается как описание API. Сервис получает сессию снаружи и коммитит сам — так одна сессия проходит через все его методы в рамках запроса.

39. Карта импортов: что откуда и почему

SQLAlchemy разложен на подпакеты по смыслу. Когда понимаешь логику, перестаёшь гуглить каждый импорт.

Откуда	Что	Смысл
sqlalchemy	select, insert, update, delete, func, and_, or_, not_, exists, case, cast, text	конструкторы SQL-выражений, ядро (Core)
sqlalchemy	Integer, String, Text, Boolean, DateTime, Date, Numeric, JSON, Enum, SmallInteger	типы колонок, общие для всех баз
sqlalchemy	Table, Column, ForeignKey, MetaData, CheckConstraint, UniqueConstraint, Index	описание схемы на уровне таблиц
sqlalchemy.orm	DeclarativeBase, Mapped, mapped_column, relationship, selectinload, joinedload	ORM: классы-модели и связи
sqlalchemy.ext.asyncio	create_async_engine, AsyncSession, async_sessionmaker	асинхронный движок и сессии
sqlalchemy.exc	IntegrityError, NoResultFound, MultipleResultsFound, SQLAlchemyError	исключения
sqlalchemy.dialects.postgresql	JSONB, UUID, ARRAY, insert (с on_conflict)	то, что есть только в PostgreSQL
alembic	op, context	команды миграций и доступ к конфигу

Разбор импортов нашей модели

backend/app/models/articles.py, шапка

```
from datetime import datetime                                # аннотации Mapped[datetime]

from sqlalchemy import (
    JSON,                # тип JSON, общий для всех баз (для SQLite в тестах)
    CheckConstraint,     # CHECK (value IN (-1, 1))
    Column,              # колонка в Table без класса-модели: article_tags
    DateTime,            # timestamp; с timezone=True — timestampz
    ForeignKey,          # внешний ключ
    Integer,             # int4
    SmallInteger,        # int2 — для value реакции хватит
    String,              # varchar(n)
    Table,               # таблица без модели — промежуточная для M:N
    Text,                # text без ограничения длины
    func,                # генератор SQL-функций: func.now(), func.count()
)
from sqlalchemy.dialects.postgresql import JSONB           # настоящий JSONB для прода
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.models.base import Base                          # общий DeclarativeBase проекта
```

- `JSON().with_variant(JSONB, 'postgresql')` — «по умолчанию JSON, а на PostgreSQL JSONB». Так одна модель работает и в тестах на SQLite, и на проде.
- `func` — не функция, а объект-генератор: любой атрибут превращается в вызов SQL-функции с тем же именем. `func.now()` даст `now()`, `func.random()` — `random()`, `func.count()` — `count(*)`. Он ничего не проверяет: опечатка уедет в базу и упадёт там.
- `Base` импортируется из одного места. У всех моделей общий `Base.metadata`, иначе Alembic увидит только часть таблиц.

Разбор импортов сервиса

backend/app/services/articles.py, шапка

```
from dataclasses import dataclass                          # ArticleStats — простой контейнер

from sqlalchemy import and_, func, select, update          # выражения и команды
from sqlalchemy.exc import IntegrityError                  # ловим гонку на уникальном ключе
from sqlalchemy.ext.asyncio import AsyncSession           # тип для аннотации self.db

from app.models.articles import Article, ArticleReaction, ArticleView, Tag, article_tags
from app.schemas.articles import ArticleCreateSchema, ArticleUpdateSchema
from app.services.content import make_slug, prepare_content
from app.services.exceptions import ArticleNotFoundError, TagNotFoundError
```

Заметьте, что сервис не импортирует ничего из FastAPI. Он не знает про HTTP, Depends и статусы ответов. Доменные ошибки — свои классы в `services/exceptions.py`, и уже роутер переводит `ArticleNotFoundError` в 404. Так сервис можно вызвать из скрипта загрузки статей или из теста.

Разбор импортов роутера

backend/app/routers/articles.py, шапка

```
from fastapi import APIRouter, Depends, HTTPException, Query, status

from app.dependencies import get_article_service, get_visitor_id
from app.schemas.articles import ArticleCardSchema, ArticleListSchema, ArticleSchema, Section
from app.services.articles import ArticleService
from app.services.exceptions import ArticleNotFoundError
```

Роутер, наоборот, не импортирует ни моделей, ни select. Он работает со схемами Pydantic на входе и выходе и с сервисом посередине. Направление зависимостей всегда одно: роутеры → сервисы → модели. Модели не знают о сервисах, сервисы не знают о роутерах.

Часть VI. Миграции Alembic

Модель описывает, какой таблица должна быть. Но база уже существует, в ней данные, и просто пересоздать таблицу нельзя. Миграция — это скрипт, который переводит базу из одного состояния схемы в другое, не теряя данных. Alembic — инструмент, который эти скрипты хранит, упорядочивает и применяет.

40. Зачем миграции и как Alembic устроен

Без миграций есть два пути, и оба плохие. `Base.metadata.create_all()` создаёт только отсутствующие таблицы и никогда не меняет существующие: добавили колонку в модель — в базе её не появится. Ручной `ALTER TABLE` в `psql` работает, но его нужно повторить на каждом окружении, помнить порядок и уметь откатить. Миграции решают обе проблемы: изменение схемы — это файл в репозитории, который применяется автоматически и одинаково везде.

Как Alembic понимает, что уже применено

В базе есть служебная таблица `alembic_version` с одной строкой — идентификатором последней применённой ревизии. Каждый файл миграции знает свой `revision` и `down_revision` (предыдущую). Из них складывается цепочка. Команда `alembic upgrade head` смотрит, где база сейчас, и применяет все ревизии от неё до конца цепочки.

Цепочка миграций проекта

```
0001_create_requests → 0002_create_articles → 0003_add_article_seo → 0004_add_article_section
      (заявки)              (статьи, теги,      (seo_title,      (колонка section
                          просмотры, реакции)  seo_description,  + индекс)
                                      seo_keywords)
```

`alembic_version` на проде после деплоя: `0004`

Структура каталога

Файлы Alembic в проекте

```
backend/
├── alembic.ini          # где лежат миграции, формат имён файлов
├── alembic/
│   ├── env.py           # как подключиться к базе и откуда взять metadata
│   ├── script.py.mako   # шаблон нового файла миграции
│   └── versions/
│       ├── 0001_create_requests.py
│       ├── 0002_create_articles.py
│       ├── 0003_add_article_seo.py
│       └── 0004_add_article_section.py
└── app/
    └── models/          # то, с чем Alembic сравнивает базу
```

41. Настройка: alembic.ini и env.py

`alembic.ini` хранит настройки самого инструмента. Из важного: путь к каталогу миграций и шаблон имени файла. URL базы мы туда не пишем, чтобы пароль не лежал в репозитории, а берём из переменных окружения в `env.py`.

alembic.ini

```
[alembic]
script_location = alembic
file_template = %(rev)s_%(slug)s
prepend_sys_path = .
```

`env.py` — Python-файл, который Alembic выполняет при каждой команде. Его задача: создать подключение и сказать, с какими моделями сравнивать базу. У нас движок асинхронный, поэтому `env.py` чуть сложнее стандартного.

alembic/env.py для асинхронного движка

```
import asyncio
from logging.config import fileConfig

from alembic import context
from sqlalchemy import pool
from sqlalchemy.ext.asyncio import async_engine_from_config

from app.config import get_settings
from app.models import Base

config = context.config
config.set_main_option("sqlalchemy.url", get_settings().database_url)

if config.config_file_name is not None:
    fileConfig(config.config_file_name)

target_metadata = Base.metadata

def run_migrations_offline() -> None:
    """Сгенерировать SQL в стандартный вывод, не подключаясь к базе."""

    context.configure(
        url=config.get_main_option("sqlalchemy.url"),
        target_metadata=target_metadata,
        literal_binds=True,
        dialect_opts={"paramstyle": "named"},
    )

    with context.begin_transaction():
        context.run_migrations()

def do_run_migrations(connection) -> None:
    """Выполнить миграции на уже открытом соединении."""

    context.configure(connection=connection, target_metadata=target_metadata)

    with context.begin_transaction():
        context.run_migrations()

async def run_migrations_online() -> None:
    """Подключиться асинхронным движком и выполнить миграции."""

    connectable = async_engine_from_config(
        config.get_section(config.config_ini_section, {}),
        prefix="sqlalchemy.",
        poolclass=pool.NullPool,
    )

    async with connectable.connect() as connection:
        await connection.run_sync(do_run_migrations)

    await connectable.dispose()

if context.is_offline_mode():
    run_migrations_offline()
else:
```

```
asyncio.run(run_migrations_online())
```

Ключевые места:

- `from app.models import Base` — импорт пакета моделей. В его `__init__.py` перечислены все модели, поэтому они регистрируются в `Base.metadata`. Забыли импортировать модель — Alembic решит, что таблицу надо удалить.
- `target_metadata = Base.metadata` — эталон, с которым autogenerate сравнивает базу.
- `connection.run_sync(do_run_migrations)` — сам Alembic синхронный, поэтому его код выполняется внутри асинхронного соединения через `run_sync`. Тот же приём, что и в тестах для `create_all`.
- `NullPool` — без пула: миграции выполняются один раз, соединение нужно одно.
- Два режима: `online` применяет миграции к базе, `offline (--sql)` печатает SQL, который был бы выполнен. Второй режим удобен для ревью.

42. Пишем миграцию руками

Файл миграции — обычный Python с двумя функциями: `upgrade` применяет изменение, `downgrade` его откатывает. Внутри используется объект `op` из `alembic` — набор команд, которые превращаются в `ALTER TABLE`, `CREATE INDEX` и так далее.

```
alembic/versions/0004_add_article_section.py
```

```
"""add section to articles
```

```
Revision ID: 0004
```

```
Revises: 0003
```

```
Create Date: 2026-09-08
```

```
"""
```

```
from typing import Sequence, Union
```

```
from alembic import op
```

```
import sqlalchemy as sa
```

```
revision: str = "0004"
```

```
down_revision: Union[str, None] = "0003"
```

```
branch_labels: Union[str, Sequence[str], None] = None
```

```
depends_on: Union[str, Sequence[str], None] = None
```

```
def upgrade() -> None:
```

```
    op.add_column(
```

```
        "articles",
```

```
        sa.Column("section", sa.String(length=16), nullable=False, server_default="blog"),
```

```
    )
```

```
    op.create_index("ix_articles_section", "articles", ["section"])
```

```
def downgrade() -> None:
```

```
    op.drop_index("ix_articles_section", table_name="articles")
```

```
    op.drop_column("articles", "section")
```

Разбор

- `revision` и `down_revision` — место в цепочке. Мы даём короткие номера вместо случайных хэшей, чтобы порядок читался по имени файла.
- `sa.Column(..., nullable=False, server_default='blog')` — главный приём при добавлении обязательной колонки в непустую таблицу. Без `server_default` PostgreSQL откажет: у существующих строк нет значения, а `NULL` запрещён. С ним все старые статьи получают «blog» в момент `ALTER`.
- `downgrade` делает ровно обратное в обратном порядке: сначала индекс, потом колонка. Индекс ссылается на колонку, поэтому первым удаляется он.
- Строковые имена таблиц и колонок, а не классы моделей. Миграция должна работать, даже когда модель уже изменилась или удалена.

Команды `op`, которые нужны чаще всего

Команда	SQL	Когда
<code>op.create_table(name, *columns)</code>	<code>CREATE TABLE</code>	новая таблица
<code>op.drop_table(name)</code>	<code>DROP TABLE</code>	откат создания
<code>op.add_column(table, sa.Column(...))</code>	<code>ALTER TABLE ... ADD COLUMN</code>	новое поле
<code>op.drop_column(table, name)</code>	<code>ALTER TABLE ... DROP COLUMN</code>	откат добавления
<code>op.alter_column(table, name, type_=..., nullable=..., new_column_name=...)</code>	<code>ALTER TABLE ... ALTER COLUMN</code>	смена типа, обязательности, имени
<code>op.create_index(name, table, [cols], unique=False)</code>	<code>CREATE INDEX</code>	индекс
<code>op.drop_index(name, table_name=...)</code>	<code>DROP INDEX</code>	откат индекса
<code>op.create_foreign_key(name, src, ref, [cols], [ref_cols], ondelete=...)</code>	<code>ADD CONSTRAINT ... FOREIGN KEY</code>	внешний ключ к существующей таблице
<code>op.create_check_constraint(name, table, condition)</code>	<code>ADD CONSTRAINT ... CHECK</code>	проверка
<code>op.create_unique_constraint(name, table, [cols])</code>	<code>ADD CONSTRAINT ... UNIQUE</code>	уникальность
<code>op.execute(sql)</code>	любой SQL	миграция данных, всё нестандартное

Миграция с данными

Иногда мало поменять схему: надо преобразовать существующие данные. Например, вынести теги из строки через запятую в отдельную таблицу. Делается это в три шага внутри одной миграции: добавить новое, перенести данные через `op.execute`, удалить старое.

Пример: колонка «минут на чтение», заполненная по существующему тексту

```
def upgrade() -> None:
    op.add_column("articles", sa.Column("reading_minutes", sa.Integer(), nullable=True))

    op.execute(
        """
        UPDATE articles
        SET reading_minutes = greatest(1, array_length(regexp_split_to_array(content, '\s+'), 1) / 180)
        """
    )

    op.alter_column("articles", "reading_minutes", nullable=False)

def downgrade() -> None:
    op.drop_column("articles", "reading_minutes")
```

Порядок важен: сначала колонка nullable, потом заполнение, потом nullable=False. Иначе ALTER упадёт на пустых значениях. В downgrade данные восстанавливать не нужно: они вычислимы заново.

Первая миграция: создание таблиц

Фрагмент 0002_create_articles.py

```
def upgrade() -> None:
    op.create_table(
        "tags",
        sa.Column("id", sa.Integer(), primary_key=True),
        sa.Column("slug", sa.String(length=100), nullable=False),
        sa.Column("title", sa.String(length=100), nullable=False),
        sa.UniqueConstraint("slug", name="uq_tags_slug"),
    )
    op.create_table(
        "article_tags",
        sa.Column("article_id", sa.Integer(), nullable=False),
        sa.Column("tag_id", sa.Integer(), nullable=False),
        sa.ForeignKeyConstraint(["article_id"], ["articles.id"], ondelete="CASCADE"),
        sa.ForeignKeyConstraint(["tag_id"], ["tags.id"], ondelete="CASCADE"),
        sa.PrimaryKeyConstraint("article_id", "tag_id"),
    )
```

Порядок создания таблиц тоже важен: article_tags ссылается на articles и tags, значит, они должны быть созданы раньше. В downgrade — обратный порядок: сначала зависимые таблицы.

43. Autogenerate и alembic check

Alembic умеет сравнить Base.metadata с реальной базой и написать миграцию сам:

Сгенерировать и применить

```
alembic revision --autogenerate -m "add reading minutes"
alembic upgrade head
```

Это удобно, но черновик нужно читать перед применением. Autogenerate надёжно замечает новые и удалённые таблицы, колонки и индексы. Он **не** замечает переименования: увидит удаление старой колонки и добавление новой, и данные потеряются. Не всегда видит смену типа и изменение server_default. И не пишет миграции данных. Поэтому мы пишем миграции руками,

а `autogenerate` используем как подсказку и как проверку.

alembic check: модели против миграций

```
alembic check
# No new upgrade operations detected.      ← модели и миграции согласованы
# New upgrade operations detected: ...      ← в моделях есть то, чего нет в миграциях
```

`alembic check` сравнивает `metadata` с базой и падает, если расхождения есть. Полезно как шаг CI: поменяли модель и забыли миграцию — сборка красная. Требуется живую базу с применёнными миграциями.

Что `autogenerate` не видит и как с этим жить

Изменение	Что сделает <code>autogenerate</code>	Что делать
переименование колонок	drop + add, данные потеряются	руками <code>op.alter_column(new_column_name=...)</code>
переименование таблицы	drop + create	руками <code>op.rename_table</code>
смена <code>server_default</code>	часто пропускает	руками <code>op.alter_column(server_default=...)</code>
<code>CheckConstraint</code>	не всегда сравнивает	руками <code>op.create_check_constraint</code>
преобразование данных	никогда	руками <code>op.execute</code>

44. Эксплуатация миграций

Команды на каждый день

```
alembic current      # какая ревизия применена к базе
alembic history      # вся цепочка
alembic upgrade head # применить всё до конца
alembic upgrade +1   # на одну ревизию вперёд
alembic downgrade -1 # на одну назад
alembic downgrade 0003 # до конкретной ревизии
alembic upgrade 0003:0004 --sql # показать SQL перехода, не выполняя
```

Как миграции применяются на проде

docker-compose.yml: миграции перед стартом сервера

```
backend:
  command: >
  sh -c "alembic upgrade head &&
        uvicorn app.main:app --host 0.0.0.0 --port 8000"
```

Контейнер бэкенда при каждом запуске сначала выполняет `alembic upgrade head` и только потом поднимает `uvicorn`. Если миграция упала, сервер не стартует, `health check` в деплое не пройдёт, и вы увидите ошибку в логах вместо тихо сломанного сайта. Схема базы всегда соответствует коду, который её использует.

Проверка перед пушем

Мы проверяли каждую миграцию офлайн, без базы: `alembic upgrade 0003:0004 --sql` печатает точный SQL. Так была найдена и подтверждена команда с `DEFAULT 'blog' NOT NULL`. Плюс е2е-тест на SQLite создаёт таблицы через `create_all` по моделям — если модель и миграция разойдутся, `alembic check` на проде это покажет.

Типичные ошибки

Симптом	Причина	Лечение
column contains null values при add_column NOT NULL	в таблице уже есть строки	server_default или сначала nullable, заполнить, потом NOT NULL
Multiple head revisions	две ветки миграций с одним down_revision (слияние веток git)	alembic merge heads, затем upgrade
Can't locate revision	файл миграции удалён, а в alembic_version его номер	вернуть файл или поправить alembic_version руками
Target database is not up to date при autogenerate	в базе не все миграции	alembic upgrade head, потом revision
autogenerate предлагает удалить таблицу	модель не импортирована в models/__init__.py	добавить импорт
MissingGreenlet в env.py	синхронный код на асинхронном движке	run_sync, как в примере выше

Правило безопасности: перед миграцией, которая удаляет или меняет тип колонки, снимите дамп (глава 12). Добавление колонок и индексов безопасно, удаление — нет.

45. Тестирование: SQLite, TestClient и подмена сессии

Прод работает на PostgreSQL, а тесты удобно гонять без сервера. Наш е2е-скрипт поднимает приложение на SQLite в файле и прогоняет через него сорок с лишним проверок: создание статей, публикацию, теги, просмотры, реакции, загрузку файлов. Вот скелет.

Скелет в2е-теста

```
import asyncio
import os
import pathlib

from fastapi.testclient import TestClient
from sqlalchemy.ext.asyncio import AsyncSession, async_sessionmaker, create_async_engine

from app.database import get_session
from app.main import app
from app.models import Base

db_file = pathlib.Path(os.environ["CHECK_DB"])
if db_file.exists():
    db_file.unlink()

engine = create_async_engine(f"sqlite+aiosqlite:/// {db_file.as_posix()}")
Session = async_sessionmaker(engine, class_=AsyncSession, expire_on_commit=False)

async def init() -> None:
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)

asyncio.run(init())

async def override_session():
    async with Session() as session:
        yield session

app.dependency_overrides[get_session] = override_session
client = TestClient(app)
ADMIN = {"X-Admin-Token": os.environ["ADMIN_API_TOKEN"]}

r = client.post("/api/v1/admin/tags", json={"title": "Экология"}, headers=ADMIN)
assert r.status_code == 201
```

Три приёма

- `Base.metadata.create_all` вместо миграций: создаёт таблицы по моделям одним вызовом. Быстро и не требует Alembic. Через `run_sync`, потому что `create_all` синхронный.
- `app.dependency_overrides[get_session] = override_session` — подмена зависимости FastAPI. Роутеры не меняются, они по-прежнему просят `get_session`, но получают тестовую сессию на SQLite.
- `TestClient` — HTTP-клиент, который вызывает приложение напрямую, без сети. Запоминает cookie между запросами, поэтому проверка «второй просмотр того же посетителя не считается» работает.

Где SQLite и PostgreSQL расходятся

Тема	PostgreSQL	SQLite	Как обошли
JSONB	есть	нет	<code>JSON().with_variant(JSONB, 'postgresql')</code>

Тема	PostgreSQL	SQLite	Как обошли
timestampz	момент времени	строка	published_at = func.now(): время ставит база
сравнение дат	по значению	по строке	даты со смещением в одном формате
now()	функция	функция	везде func.now(), а не datetime.now()
UUID	тип uuid	нет	String(36)
ON CONFLICT	есть	есть (с 3.24)	не используем, ловим IntegrityError
secure cookie по http	не про базу	—	COOKIE_SECURE=false в тестовом окружении

Вывод: тесты на SQLite ловят логические ошибки, ошибки типов и потоки данных, но не гарантируют, что SQL поедет на PostgreSQL. Поэтому перед пушем мы дополнительно смотрели SQL миграций офлайн, а деплой падает, если alembic upgrade head не прошёл. Для серьёзного проекта следующий шаг — тесты на настоящем PostgreSQL в Docker, например через библиотеку testcontainers.

Часть VII. Практика

46. Как проектировать схему

Порядок действий, который экономит недели переделок.

Шаг 1. Выписать сущности и их поля

Существительные из задачи: заявка, статья, тег, заказ, отклик. Для каждой — какие данные о ней нужны.

Шаг 2. Определить связи

Для каждой пары сущностей задать вопрос с обеих сторон: сколько Б у одного А и сколько А у одного Б. Ответ даёт тип связи и место внешнего ключа.

Шаг 3. Выбрать типы

- Идентификаторы (ИНН, телефон, артикул) — строкой.
- Деньги — Numeric или целое в копейках.
- Длинный текст — Text.
- Фиксированный набор — Enum.
- Время — всегда с `timezone=True`.

Шаг 4. Расставить ограничения

Что база **не должна** позволить записать? Каждое такое правило — `unique`, `CheckConstraint` или `NOT NULL`. Проверки в коде не заменяют ограничений в схеме: код можно обойти, схему — нет.

Шаг 5. Спланировать индексы

Выписать запросы, которые будут выполняться часто, и посмотреть, по каким колонкам идёт фильтрация и сортировка. Внешние ключи индексировать почти всегда.

Шаг 6. Решить, что делать при удалении

Для каждой связи: удалять детей (`CASCADE`), обнулять ссылку (`SET NULL`) или запрещать удаление (`RESTRICT`).

Денормализация — осознанное решение, а не небрежность. Счётчики `views_count` и `likes_count` дублируют то, что можно посчитать запросом. Но список из 12 карточек потребовал бы 36 подзапросов `count`. Держим агрегат в строке и обновляем его в той же транзакции, что и детальную запись. Правило: сначала нормальная схема, денормализация — когда измерили и увидели проблему.

47. Частые ошибки

MissingGreenlet

Обращение к незагруженной связи. Лечится `lazy="selectin"` в модели или `selectinload` в запросе. Вторая причина — чтение атрибутов после `commit` без `expire_on_commit=False`.

Пустая миграция

Модель не импортирована в `models/__init__.py`, значит её нет в `metadata`.

Колонка не появилась

```
published_at: DateTime(timezone=True) # это не колонка
published_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True))
```

Потерянные обновления счётчика

```
article.views_count = article.views_count + 1 # гонка
update(...).values(views_count=Article.views_count + 1) # правильно
```

Дубли при вставке под нагрузкой

Проверка перед вставкой не защищает от гонок — нужен `UNIQUE` плюс `on_conflict_do_nothing`.

Дубли в выборке после JOIN

У статьи несколько тегов — `JOIN` вернёт её несколько раз. Используйте `.any()` вместо `join` для фильтрации.

SUM вернул None

Агрегат по пустой выборке даёт `NULL`. Оборачивайте в `func.coalesce(..., 0)`.

Запрос не изменился

```
stmt.where(Article.published_at.is_not(None)) # результат выброшен
stmt = stmt.where(Article.published_at.is_not(None)) # правильно
```

48. Шпаргалка

Выбор связи

один Б у А и один А у Б	-> 1:1	relationship(uselist=False) + unique=True на FK
много Б у А, один А у Б	-> 1:N	FK в таблице «многих» relationship(back_populates=...)
много Б у А и много А у Б	-> M:N	Table(...) + secondary=... или association object, если у связи есть свои поля

Обязательный минимум для async

```
async_sessionmaker(engine, expire_on_commit=False) # иначе падёт после commit
relationship(..., lazy="selectin")                # иначе MissingGreenlet
ForeignKey(..., ondelete="CASCADE") + passive_deletes=True
mapped_column(ForeignKey(...), index=True)         # FK не индексируется сам
```

Частые запросы

await session.get(Model, pk)	# по первичному ключу
select(M).where(M.f == v)	# фильтр
select(M).where(M.rel.any(R.f == v))	# по связи, без дублей
select(M).options(selectinload(M.rel))	# загрузить связь
select(func.count()).select_from(M)	# количество
select(func.coalesce(func.sum(M.f), 0))	# сумма без None
update(M).where(...).values(f=M.f + 1)	# атомарный инкремент
insert(M).values(...).on_conflict_do_nothing()	# вставка без дублей

Чек-лист модели

- Каждая колонка — имя: Mapped[тип] = mapped_column(...)
- Нет index=True на primary_key
- Нет дублирования nullable и аннотации
- У внешних ключей есть index=True и ondelete
- Идентификаторы (ИНН, телефон) — строками
- Длинный текст — Text
- Время — DateTime(timezone=True)
- Ограничения именованные (name=)
- Модель импортирована в models/__init__.py
- Класс в единственном числе, таблица во множественном

Проверка перед собеседованием. Объясните вслух: чем Mapped отличается от mapped_column; почему для M:N нужна третья таблица; что такое N+1 и как его увидеть; чем cascade отличается от ondelete; зачем flush, если есть commit; почему проверка перед вставкой не защищает от дублей; когда WHERE, а когда HAVING.

Часть VIII. Вопросы с собеседований

49. Топ-50 вопросов, коротких ответов и примеров

Формат: вопрос, ответ на две-пять фраз, который можно произнести вслух, пример кода из проекта и ссылка на главу с подробностями. Отвечать заученно не нужно: интервьюер ждёт, что вы назовёте суть и покажете, где применили. Примеры здесь — из *pedra*, ими и пользуйтесь.

Веб-сервер и асинхронность

1. Что такое WSGI и ASGI и чем они отличаются? → глава 37

Это два стандарта интерфейса между веб-сервером и Python-приложением. WSGI — синхронный: сервер вызывает функцию приложения, та возвращает ответ, один поток обслуживает один запрос; на нём Flask и классический Django. ASGI — асинхронный наследник: приложение — *async*-функция, которая отдаёт управление на время ожидания, поэтому один процесс держит много запросов, плюс WebSocket. FastAPI — ASGI-приложение.

```
# WSGI: одна функция, синхронно
def app(environ, start_response):
    start_response("200 OK", [("Content-Type", "text/plain")])
    return [b"hello"]

# ASGI: корутина, три аргумента
async def app(scope, receive, send):
    await send({"type": "http.response.start", "status": 200, "headers": []})
    await send({"type": "http.response.body", "body": b"hello"})
```

2. Зачем нужен uvicorn, если есть FastAPI?

FastAPI — приложение: маршруты, валидация, зависимости. Слушать порт и разбирать HTTP оно не умеет, это делает ASGI-сервер *uvicorn*: принимает соединения, парсит запросы, вызывает приложение, отправляет ответы. Перед ним стоит *nginx*: TLS, статика, прокси.

```
# docker-compose.yml, сервис backend
command: >
  sh -c "alembic upgrade head &&
    uvicorn app.main:app --host 0.0.0.0 --port 8000"

# app.main:app — путь к объекту FastAPI: модуль app/main.py, переменная app
```

3. Что такое event loop и что делает await? → глава 37

Event loop — цикл, который выполняет задачи в одном потоке по очереди. На *await* операции ввода-вывода задача говорит «я жду», цикл переключается на другую; когда данные пришли, первая продолжается с того же места. Код между двумя *await* выполняется без прерываний.

```
import asyncio

async def fetch(name: str, seconds: float) -> str:
    await asyncio.sleep(seconds)      # здесь цикл занимается другими задачами
    return name

async def main() -> None:
```

```

results = await asyncio.gather(fetch("a", 1), fetch("b", 1), fetch("c", 1))
print(results)                                # ['a', 'b', 'c'] через ~1 секунду, а не 3

asyncio.run(main())

```

4. Что такое GIL и почему асинхронность помогает, несмотря на него?

GIL — глобальная блокировка интерпретатора: байткод Python в один момент выполняет один поток, поэтому потоки не ускоряют вычисления. Но бэкенд в основном ждёт базу и сеть, а во время ожидания GIL не нужен. Асинхронность заполняет паузы другой работой. Для процессорных задач нужны процессы.

```

# ждёт сеть — асинхронность помогает
articles = await db.execute(select(Article))

# считает — GIL мешает, асинхронность не поможет
result = sum(i * i for i in range(50_000_000))
# выносить в процесс: ProcessPoolExecutor или отдельный воркер

```

5. Чем в FastAPI отличается ручка def от async def? → глава 37

async def выполняется в event loop, внутри можно только неблокирующие вызовы. Обычную def FastAPI запускает в пуле потоков, чтобы она не останавливала цикл; там можно блокирующий код, но пул ограничен. Блокирующий вызов внутри async def останавливает всё приложение.

```

@router.get("/articles/{slug}")
async def get_article(slug: str, service: ArticleService = Depends(get_article_service)):
    article = await service.get_published(slug)      # asyncpg, не блокирует
    return ArticleSchema.model_validate(article)

@router.get("/report")
def build_report():                                # без async: уйдёт в пул потоков
    return heavy_sync_library.render()              # блокирующий код здесь допустим

```

6. Что случится, если внутри async def вызвать time.sleep(5)?

Event loop заблокируется на пять секунд: другие запросы не обрабатываются, health check не отвечает. В асинхронном коде используют asyncio.sleep, асинхронные драйверы, а тяжёлую синхронную работу выносят в пул потоков.

```

import asyncio
from fastapi.concurrency import run_in_threadpool

async def handler():
    time.sleep(5)                                # плохо: встал весь сервер
    await asyncio.sleep(5)                       # хорошо: ждём, не блокируя
    data = await run_in_threadpool(parse_big_pdf, path) # синхронную функцию — в поток

```

7. Сколько воркеров uvicorn запускать? → глава 37

Один воркер — один процесс с одним event loop, одно ядро. Правило: число воркеров около числа ядер. У каждого свой пул соединений, поэтому суммарно workers × pool_size не должно превышать лимит PostgreSQL, по умолчанию сто.

```

uvicorn app.main:app --host 0.0.0.0 --port 8000 --workers 4

# 4 воркера × pool_size 5 + max_overflow 10 = до 60 соединений
# PostgreSQL max_connections по умолчанию 100 — запас есть

```

8. Зачем BackgroundTasks и чем они отличаются от Celery?

BackgroundTasks выполняет функцию после отправки ответа, в том же процессе: быстро и без инфраструктуры, но без очереди и повторов. Celery — отдельные воркеры с брокером: надёжно, с повторами и расписанием, но сложнее. Начинают с первого.

```
@router.post("/request", status_code=201)
async def create_request(
    payload: RequestCreateSchema,
    background: BackgroundTasks,
    service: RequestService = Depends(get_request_service),
) -> RequestOutSchema:
    request = await service.create(payload)
    background.add_task(send_request_email, request) # клиент ответ уже получил
    return RequestOutSchema.model_validate(request)
```

9. Как работает Depends и когда вызываются зависимости? → глава 38

Перед ручкой FastAPI строит дерево зависимостей и вызывает каждую один раз на запрос, результат кэшируется. Зависимость с `yield` получает «после»: код после `yield` выполняется, когда ответ отправлен. Зависимости можно вешать на весь роутер.

```
async def get_session() -> AsyncGenerator[AsyncSession, None]:
    async with SessionLocal() as session:
        yield session
        # ← ручка работает здесь
        # ← после ответа: сессия закрыта
```

```
router = APIRouter(prefix="/admin", dependencies=[Depends(require_admin)])
# каждая ручка роутера сначала пройдёт проверку токена
```

10. Опишите путь запроса от браузера до базы в вашем проекте. → глава 38

Браузер → nginx (TLS, статика, /api/ проксируется) → uvicorn → FastAPI: роутер, валидация Pydantic, зависимости создают сессию и сервис → сервис строит запрос SQLAlchemy → asyncpg шлёт SQL в PostgreSQL → объекты модели → схема ответа → JSON. Для страниц сайта перед этим ещё Next.js, который ходит в тот же API с сервера.

```
# deploy/nginx/nedra.conf
location /api/ { proxy_pass http://backend:8000; }
location /media/ { alias /var/www/media/; expires 30d; }
location / { proxy_pass http://frontend:3000; }
```

PostgreSQL

11. Расшифруйте ACID. → глава 11

Atomicity — целиком или откат. Consistency — после транзакции все ограничения выполнены. Isolation — параллельные транзакции не видят промежуточных состояний друг друга. Durability — после COMMIT данные не потеряются. Лайк у нас — вставка и увеличение счётчика в одной транзакции.

```
BEGIN;
INSERT INTO article_reactions (article_id, visitor_id, value) VALUES (42, 'x', 1);
UPDATE articles SET likes_count = likes_count + 1 WHERE id = 42;
COMMIT; -- обе строки видны одновременно; упало посередине — не видна ни одна
```

12. Какие уровни изоляции есть и какой по умолчанию? → глава 11

Read Committed (по умолчанию): каждый запрос видит закоммиченное к его началу. Repeatable Read: снимок на момент начала транзакции. Serializable: как будто по очереди, при конфликте откат. Read Uncommitted в PostgreSQL ведёт себя как Read Committed. Чем строже, тем больше откатов.

```
BEGIN ISOLATION LEVEL REPEATABLE READ;
SELECT sum(likes_count) FROM articles;  -- оба SELECT увидят один снимок,
SELECT count(*) FROM article_reactions; -- даже если параллельно кто-то коммитит
COMMIT;
```

13. Что такое MVCC? → глава 12

Multi-Version Concurrency Control: UPDATE не перезаписывает строку, а создаёт новую версию; старую видят транзакции, начавшиеся раньше. Читатели не блокируют писателей. Плата — мёртвые версии, которые убирает VACUUM.

```
-- скрытые системные колонки показывают версии строк
SELECT xmin, xmax, id, likes_count FROM articles WHERE id = 42;
-- xmin: транзакция, создавшая версию; xmax: транзакция, её удалившая (0 — живая)
```

14. Что такое грязное, неповторяющееся и фантомное чтение? → глава 11

Грязное — увидеть незакоммиченное; в PostgreSQL невозможно. Неповторяющееся — одна строка дважды прочитана с разными значениями внутри транзакции; возможно на Read Committed. Фантомное — повторный запрос по условию вернул другое число строк. Repeatable Read закрывает оба последних.

```
-- сессия 1 (Read Committed)      -- сессия 2
BEGIN;
SELECT likes_count FROM articles
WHERE id = 42;                    -- 10

                                UPDATE articles SET likes_count = 11 WHERE id = 42;
                                COMMIT;

SELECT likes_count FROM articles
WHERE id = 42;                    -- 11 ← неповторяющееся чтение
COMMIT;
```

15. Что такое дедлок и как его избежать? → глава 11

Две транзакции ждут друг друга: первая держит строку А и хочет В, вторая наоборот. PostgreSQL находит цикл и откатывает одну. Профилактика: обновлять строки в одном порядке, держать транзакции короткими, не делать сетевых вызовов внутри.

```
-- сессия 1                      -- сессия 2
UPDATE articles SET ... WHERE id = 1;  UPDATE articles SET ... WHERE id = 2;
UPDATE articles SET ... WHERE id = 2;  UPDATE articles SET ... WHERE id = 1;
-- ждёт сессию 2                  -- ждёт сессию 1 → ERROR: deadlock detected

-- лечение: всегда в порядке возрастания id
UPDATE articles SET ... WHERE id IN (1, 2) ORDER BY id; -- или два UPDATE по порядку
```

16. Какие типы индексов есть в PostgreSQL? → глава 5

B-tree по умолчанию: равенство, диапазоны, сортировка. Hash — только равенство. GIN — «содержит»: JSONB, массивы, полнотекст. GiST — геометрия, диапазоны. BRIN — огромные

таблицы с естественным порядком. Плюс уникальные, составные, частичные и функциональные.

```
CREATE INDEX ix_articles_published_at ON articles (published_at);           -- B-tree
CREATE INDEX ix_articles_toc ON articles USING GIN (toc);                   -- JSONB
CREATE UNIQUE INDEX uq_tags_slug ON tags (slug);                           -- уникальный
CREATE INDEX ix_articles_live ON articles (published_at) WHERE published_at IS NOT NULL; -- частичный
CREATE INDEX ix_articles_title_lower ON articles (lower(title));            -- функциональный
```

17. Когда индекс не используется, хотя он есть? → глава 5

Маленькая таблица — перебор дешевле. Условие не совпадает с индексом: процент в начале LIKE, функция над колонкой без функционального индекса, приведение типа. Составной индекс начинается не с той колонки. Низкая селективность. Проверка — EXPLAIN.

```
WHERE title LIKE '%надзор%'          -- индекс по title бесполезен: неизвестно начало
WHERE lower(title) = 'ovos'          -- нужен индекс по lower(title)
WHERE published_at::date = '2026-09-07' -- приведение ломает индекс по published_at
WHERE published_at >= '2026-09-07' AND published_at < '2026-09-08' -- так индекс работает
```

18. Как выбрать порядок колонок в составном индексе? → глава 5

Сначала колонки с равенством, потом колонка диапазона или сортировки. Индекс по (a, b) работает для запросов по a, но не только по b.

```
-- запрос
SELECT * FROM articles
WHERE section = 'news' AND published_at <= now()
ORDER BY published_at DESC LIMIT 12;
```

```
CREATE INDEX ix_articles_section_published ON articles (section, published_at DESC);
-- равенство по section → блок раздела, внутри уже отсортировано по дате
```

19. Как читать EXPLAIN ANALYZE? → глава 5

Снизу вверх, изнутри наружу. Смотрим тип узла (Seq Scan, Index Scan, Hash Join), оценку строк против фактической и время. Большое расхождение — устаревшая статистика, нужен ANALYZE. Для UPDATE оборачивать в транзакцию с ROLLBACK.

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM articles WHERE section = 'news' ORDER BY published_at DESC LIMIT 12;
```

```
Limit (actual time=0.05..0.09 rows=12)
-> Index Scan Backward using ix_articles_published_at on articles
   Filter: (section = 'news')
   Rows Removed by Filter: 2          ← сколько прочитали зря
```

20. Что такое нормализация и когда её нарушают? → глава 4

Каждый факт хранится один раз: название тега в tags, связь в article_tags. 1НФ — нет списков в ячейке, 2НФ — нет зависимостей от части ключа, 3НФ — нет зависимостей между неключевыми колонками. Нарушают ради скорости чтения: наши счётчики — денормализация.

```
-- нарушение 1НФ: список в ячейке, не поискать и не проверить
tags TEXT -- 'промбез, экология, опо'
```

```
-- нормально: промежуточная таблица
CREATE TABLE article_tags (
  article_id INTEGER REFERENCES articles(id) ON DELETE CASCADE,
  tag_id      INTEGER REFERENCES tags(id)      ON DELETE CASCADE,
```

```

PRIMARY KEY (article_id, tag_id)
);

-- осознанная денормализация: счётчик вместо count(*) по реакциям на каждый показ
likes_count INTEGER NOT NULL DEFAULT 0

```

21. Чем PRIMARY KEY отличается от UNIQUE? → глава 4

Оба гарантируют уникальность и создают индекс. Первичный ключ один, не допускает NULL и идентифицирует строку, на него ссылаются внешние ключи. UNIQUE-ограничений много, и NULL там допускается сколько угодно раз.

```

CREATE TABLE articles (
    id SERIAL PRIMARY KEY,          -- один, NOT NULL, цель внешних ключей
    slug VARCHAR(255) NOT NULL UNIQUE, -- второй уникальный признак
    ...
);
INSERT INTO tags (slug, title) VALUES (NULL, 'a'), (NULL, 'b'); -- UNIQUE пропустит оба NULL

```

22. Что делает внешний ключ и какие варианты ON DELETE бывают? → глава 4

Гарантирует, что ссылка указывает на существующую строку, и задаёт поведение при удалении родителя: RESTRICT, CASCADE, SET NULL, SET DEFAULT. Индекс на колонку внешнего ключа надо создавать вручную.

```

article_id INTEGER NOT NULL REFERENCES articles(id) ON DELETE CASCADE
-- удалили статью → её реакции и просмотры исчезли

executor_id INTEGER REFERENCES users(id) ON DELETE SET NULL
-- удалили пользователя → заказ остался без исполнителя

INSERT INTO article_tags VALUES (999, 1);
-- ERROR: violates foreign key constraint — статьи 999 нет

```

23. Что такое VACUUM и зачем он нужен? → глава 12

После UPDATE и DELETE остаются мёртвые версии строк (MVCC). VACUUM освобождает их место для повторного использования и обновляет статистику планировщика. Autovacuum делает это сам. VACUUM FULL сжимает файл, но блокирует таблицу.

```

VACUUM ANALYZE articles;          -- убрать мёртвые версии, обновить статистику

SELECT relname, n_dead_tup, last_autovacuum
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC;         -- где накопился мусор

```

24. Когда хранить данные в JSONB, а когда в отдельной таблице? → глава 3

JSONB — для переменной структуры, которая читается целиком и на которую не ссылаются: оглавление, настройки, сырой ответ API. Таблица — когда нужно искать, соединять, агрегировать, ограничивать уникальность и ссылаться внешними ключами.

```

-- оглавление: читается целиком вместе со статьёй → JSONB
toc JSONB -- [{"id": "что-меняется", "title": "Что меняется"}, ...]

SELECT title FROM articles WHERE toc @> '[{"id": "riski"}]'; -- поиск по GIN-индексу

```

```
SELECT title, toc->0->>'title' AS first_h2 FROM articles;      -- достать поле

-- теги: нужны фильтр, уникальность, каскад → таблица, не JSONB
```

25. Чем timestamp отличается от timestamptz? → глава 3

timestamp хранит настенное время без пояса. timestamptz — момент времени, внутри UTC, при показе переводится в пояс сессии. Правило: всегда timestamptz, в SQLAlchemy `DateTime(timezone=True)`.

```
SELECT '2026-09-07 09:00+07':timestamptz;  -- 2026-09-07 02:00:00+00 в сессии UTC
SET TIME ZONE 'Asia/Novosibirsk';
SELECT '2026-09-07 09:00+07':timestamptz;  -- 2026-09-07 09:00:00+07 — то же значение
```

```
published_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True), index=True)
```

26. Как делаются бэкапы PostgreSQL? → глава 12

`pg_dump` — логический дамп в SQL, просто восстанавливается. Физический бэкап плюс архив WAL даёт восстановление на любой момент времени. Репликация — копия на другом сервере для отказоустойчивости, бэкап не заменяет.

```
docker compose exec -T db sh -c 'pg_dump -U "$POSTGRES_USER" -d "$POSTGRES_DB" --clean' \
> backup-$(date +%F).sql
```

```
docker compose exec -T db sh -c 'psql -U "$POSTGRES_USER" -d "$POSTGRES_DB"' < backup-2026-09-09.sql
```

SQL

27. Какие виды JOIN бывают? → глава 7

INNER — только совпавшие пары. LEFT — все строки слева, справа NULL без пары. RIGHT — зеркально. FULL — всё с обеих сторон. CROSS — каждая с каждой.

```
-- статьи с числом тегов, включая статьи без тегов
SELECT a.slug, count(at.tag_id) AS tags_count
FROM articles a
LEFT JOIN article_tags at ON at.article_id = a.id
GROUP BY a.id, a.slug;
```

```
-- с INNER JOIN статьи без тегов пропали бы
```

28. Чем WHERE отличается от HAVING? → глава 8

WHERE фильтрует строки до группировки и не видит агрегатов. HAVING фильтрует группы после GROUP BY. Что можно выразить в WHERE — туда: меньше строк дойдёт до группировки.

```
SELECT t.title, count(*) AS articles
FROM tags t
JOIN article_tags at ON at.tag_id = t.id
JOIN articles a      ON a.id = at.article_id
WHERE a.published_at <= now()      -- до группировки: только опубликованные
GROUP BY t.id, t.title
HAVING count(*) >= 5               -- после: только популярные теги
ORDER BY articles DESC;
```

29. В чём разница между count(*), count(колонка) и count(DISTINCT)? →

глава 8

count(*) — строки. count(колонка) — строки, где колонка не NULL. count(DISTINCT колонка) — число разных значений.

```
SELECT
    count(*)                AS total,          -- 100
    count(cover_image)      AS with_cover,     -- 100 (NULL не считается)
    count(DISTINCT section) AS sections,      -- 2
    count(*) FILTER (WHERE published_at > now()) AS scheduled
FROM articles;
```

30. Подзапрос, JOIN или EXISTS — что выбрать? → глава 9

Нужны колонки второй таблицы — JOIN. Нужно только проверить наличие — EXISTS: останавливается на первой строке и не размножает результат. IN читается проще, но на больших списках медленнее.

```
-- SQL
SELECT slug FROM articles a
WHERE EXISTS (SELECT 1 FROM article_tags at JOIN tags t ON t.id = at.tag_id
              WHERE at.article_id = a.id AND t.slug = 'ekologiya');
```

```
# SQLAlchemy: то же самое одной строкой
stmt = select(Article).where(Article.tags.any(Tag.slug == tag_slug))
```

31. Что такое оконные функции? → глава 9

Считают значение по группе строк, не сворачивая их: каждая строка остаётся, рядом появляется row_number(), sum() OVER, lag(). PARTITION BY делит на группы, ORDER BY задаёт порядок внутри окна.

```
SELECT slug, section, views_count,
       row_number() OVER (PARTITION BY section ORDER BY views_count DESC) AS place,
       sum(views_count) OVER (PARTITION BY section) AS section_views
FROM articles;
```

```
# SQLAlchemy
from sqlalchemy import func, over
place = func.row_number().over(partition_by=Article.section, order_by=Article.views_count.desc())
stmt = select(Article.slug, place.label("place"))
```

32. Как сделать «вставить или обновить» одним запросом? → глава 10

INSERT ... ON CONFLICT (колонки) DO UPDATE или DO NOTHING. Конфликт определяется по уникальному ограничению. Атомарно, снимает гонку «проверил — вставил».

```
INSERT INTO article_reactions (article_id, visitor_id, value)
VALUES (42, 'x', 1)
ON CONFLICT (article_id, visitor_id) DO UPDATE SET value = EXCLUDED.value;
```

```
# SQLAlchemy: диалектный insert
from sqlalchemy.dialects.postgresql import insert
```

```
stmt = insert(ArticleReaction).values(article_id=42, visitor_id="x", value=1)
stmt = stmt.on_conflict_do_update(
    index_elements=[ArticleReaction.article_id, ArticleReaction.visitor_id],
    set_={"value": stmt.excluded.value},
```



```
)
await db.execute(stmt)
```

SQLAlchemy

33. Что такое Core и ORM в SQLAlchemy? → глава 13

Core — конструктор SQL: select, Table, результат — строки. ORM надстроен над Core: классы-модели, объекты, сессия с identity map и unit of work. В 2.0 у них один язык запросов.

```
# Core: таблица без класса, результат — строки
article_tags = Table("article_tags", Base.metadata, Column(...), Column(...))
rows = await db.execute(select(article_tags.c.tag_id).where(article_tags.c.article_id == 42))

# ORM: класс, результат — объекты
class Tag(Base):
    __tablename__ = "tags"
    id: Mapped[int] = mapped_column(primary_key=True)

tags = (await db.execute(select(Tag))).scalars().all()
```

34. Engine, Connection, Session — кто за что отвечает? → глава 37

Engine — один на приложение: URL, диалект, пул. Connection — одно соединение из пула, уровень Core. Session — рабочее пространство ORM: берёт соединение, ведёт транзакцию, хранит объекты.

```
engine = create_async_engine(settings.database_url, pool_pre_ping=True) # один раз
SessionLocal = async_sessionmaker(engine, class_=AsyncSession, expire_on_commit=False)

async with SessionLocal() as db: # сессия на запрос
    article = await db.get(Article, 42) # соединение взято из пула здесь

async with engine.connect() as conn: # чистый Core без ORM, редко
    await conn.execute(text("SELECT 1"))
```

35. Что такое identity map? → глава 38

Словарь «первичный ключ → объект» внутри сессии: одна строка — один объект Python. Повторный get не ходит в базу. Отсюда ловушка: после commit объект в карте остался, а вычисленные базой колонки протухли.

```
first = await db.get(Article, 42)
second = await db.get(Article, 42)
assert first is second # тот же объект, второго запроса не было

await db.commit()
again = await db.get(Article, 42) # снова тот же объект, updated_at устарел
stmt = select(Article).where(Article.id == 42).execution_options(populate_existing=True)
fresh = (await db.execute(stmt)).scalar_one() # перечитали и перезаписали объект
```

36. Чем flush отличается от commit? → глава 38

flush отправляет накопленные изменения в базу внутри открытой транзакции: появляется id, ловится IntegrityError. commit делает flush и фиксирует транзакцию.

```
self.db.add(ArticleView(article_id=article.id, visitor_id=visitor_id))
```

```

try:
    await self.db.flush()                # INSERT ушёл; дубль по первичному ключу упадёт здесь
except IntegrityError:
    await self.db.rollback()
    return                               # просмотр уже учтён параллельным запросом

await self.db.execute(update(Article).where(Article.id == article.id)
                      .values(views_count=Article.views_count + 1))
await self.db.commit()                  # оба изменения зафиксированы вместе

```

37. Что такое `expire_on_commit` и почему в `async` его выключают? → глава 38

После `commit` сессия по умолчанию помечает атрибуты устаревшими, следующее обращение делает запрос. В `async` обращение к атрибуту — не `await`, запрос невозможен → `MissingGreenlet`. Поэтому `expire_on_commit=False`, а вычисленные базой колонки перечитываем явно.

```
SessionLocal = async_sessionmaker(engine, class_=AsyncSession, expire_on_commit=False)
```

```

await db.commit()
print(article.title)          # ок: атрибут не протух
print(article.updated_at)     # onupdate=func.now() — значение знает только база → reload()

```

38. Что такое `N+1` и как его увидеть? → глава 27

Загрузили `N` статей одним запросом, потом для каждой обратились к связи — ещё `N` запросов. Видно через `echo=True`: десятки одинаковых `SELECT`. Лечится стратегией загрузки.

```

# плохо: 1 запрос за статьями + по запросу на теги каждой
articles = (await db.execute(select(Article))).scalars().all()
for article in articles:
    print(article.tags)          # в async это ещё и MissingGreenlet

# хорошо: теги одним дополнительным запросом
stmt = select(Article).options(selectinload(Article.tags))
# или раз и навсегда в модели:
tags: Mapped[list["Tag"]] = relationship(secondary=article_tags, lazy="selectin")

```

39. `selectinload` или `joinedload`? → глава 27

`joinedload` — один запрос с `JOIN`, хорош для связей «к одному». `selectinload` — второй запрос по списку `id`, лучше для коллекций: строки не дублируются. В `async` ленивой загрузки нет, стратегию задают явно.

```

-- selectinload: два запроса
SELECT * FROM articles WHERE ... LIMIT 12;
SELECT * FROM article_tags JOIN tags ... WHERE article_tags.article_id IN (1, 2, ..., 12);

-- joinedload: один запрос, строки статей повторяются по числу тегов
SELECT articles.*, tags.* FROM articles LEFT JOIN article_tags ... LEFT JOIN tags ...;

# к одному — joinedload
stmt = select(Order).options(joinedload(Order.customer))

```

40. Чем `cascade` у `relationship` отличается от `ondelete` у `ForeignKey`? → глава 28

`ondelete='CASCADE'` — правило базы, работает при любом удалении и одним запросом.
`cascade='all, delete-orphan'` — правило ORM: только при удалении через сессию и только для загруженных объектов. Надёжнее полагаться на базу.

```
# база: работает даже из psycopg
article_id: Mapped[int] = mapped_column(ForeignKey("articles.id", ondelete="CASCADE"))

# ORM: сессия сама удалит детей, если удаляем через неё
items: Mapped[list["Item"]] = relationship(cascade="all, delete-orphan")

await db.delete(article)      # ORM удалит статью; связи снесёт база каскадом
await db.commit()
```

41. Почему сессия создаётся на каждый запрос, а не одна на приложение? → глава 38

Сессия — это транзакция и состояние объектов. Общая сессия смешала бы данные разных пользователей, не потокобезопасна, копила бы объекты и держала транзакцию вечно. Сессия на запрос даёт изоляцию и автоматическую уборку.

```
# плохо
db = SessionLocal()                # одна на модуль

# хорошо: зависимость с yield
async def get_session():
    async with SessionLocal() as session:
        yield session

def get_article_service(session: AsyncSession = Depends(get_session)) -> ArticleService:
    return ArticleService(session)
```

42. Как подобрать размер пула соединений? → глава 37

Столько, сколько запросов реально ждут базу одновременно, обычно 5–20 на воркер. Больше — не быстрее: PostgreSQL ограничен ядрами и `max_connections`. Считать суммарно по воркерам и сервисам.

```
engine = create_async_engine(
    url,
    pool_size=5,          # постоянные
    max_overflow=10,       # временные под пик
    pool_pre_ping=True,    # проверять живость перед выдачей
    pool_recycle=1800,     # пересоздавать старые
)
# симптом нехватки: sqlalchemy.exc.TimeoutError: QueuePool limit of size 5 overflow 10 reached
```

43. Когда лучше написать сырой SQL вместо ORM? → глава 31

Сложная аналитика с окнами и CTE, массовые обновления, диалектные возможности. Через `text()` с параметрами, а не f-строками — иначе SQL-инъекция.

```
from sqlalchemy import text

stmt = text("""
    WITH scheduled AS (
        SELECT id, row_number() OVER (ORDER BY published_at) AS n
        FROM articles WHERE section = :section AND published_at > now()
    )
    UPDATE articles a SET published_at = :start - scheduled.n * interval '1 day'
""")
```

```
FROM scheduled WHERE a.id = scheduled.id
""")
await db.execute(stmt, {"section": "news", "start": start}) # параметры, не f-строка
```

44. Зачем миграции, если есть `create_all`? Что не умеет `autogenerate`? →

глава 43

`create_all` только создаёт отсутствующие таблицы и не меняет существующие. Миграции — версионированные шаги с откатом. `Autogenerate` не видит переименований, не всегда видит смену типа и `default`, не пишет миграции данных.

```
def upgrade() -> None:
    op.add_column("articles",
        sa.Column("section", sa.String(16), nullable=False, server_default="blog"))
    op.create_index("ix_articles_section", "articles", ["section"])

def downgrade() -> None:
    op.drop_index("ix_articles_section", table_name="articles")
    op.drop_column("articles", "section")
```

`server_default` обязателен: в таблице уже 100 строк, NOT NULL без него упадёт

45. Что такое `Mapped` и `mapped_column` и как определяется `nullable`? →

глава 14

`Mapped[T]` — аннотация: тип и обязательность. `Mapped[str]` даёт NOT NULL, `Mapped[str | None]` разрешает NULL. `mapped_column` добавляет то, чего нет в типе: длину, индекс, `default`, внешний ключ.

```
title: Mapped[str] = mapped_column(String(255)) # NOT NULL
description: Mapped[str | None] = mapped_column(String(400)) # NULL разрешён
views_count: Mapped[int] = mapped_column(Integer, default=0) # default в Python
created_at: Mapped[datetime] = mapped_column(
    DateTime(timezone=True), server_default=func.now() # default в базе
)
```

Архитектура

46. Зачем слой сервисов между роутером и моделями? → глава 38

Роутер отвечает за HTTP, сервис — за данные и правила. Логику можно вызвать из скриптов и тестов без HTTP, роутеры короткие, зависимости идут в одну сторону: роутеры → сервисы → модели.

```
# сервис: не знает про HTTP
async def get_published(self, slug: str) -> Article:
    result = await self.db.execute(select(Article).where(Article.slug == slug, PUBLISHED))
    article = result.scalar_one_or_none()
    if article is None:
        raise ArticleNotFoundError(f"Статья «{slug}» не найдена")
    return article

# роутер: только HTTP
try:
    article = await service.get_published(slug)
except ArticleNotFoundError as error:
    raise HTTPException(status.HTTP_404_NOT_FOUND, str(error)) from error
```

47. Где ловить ошибки и как превращать их в HTTP-ответы? → глава 39

Сервис бросает доменные исключения, роутер переводит их в `HTTPException` с кодом. Валидацию входа делает Pydantic (422). Непредвиденные исключения не ловим — пусть будет 500 с записью в лог.

```
# services/exceptions.py
class ArticleNotFoundError(LookupError):
    """Статья не найдена или не опубликована."""

class UploadError(ValueError):
    """Файл не подходит: неверный тип или превышен размер."""

# роутер
except UploadError as error:
    raise HTTPException(status.HTTP_422_UNPROCESSABLE_ENTITY, str(error)) from error
```

48. Зачем ограничения в базе, если есть валидация Pydantic? → глава 4

Pydantic защищает вход API. База защищает данные от всех источников: миграций, скриптов, ручных правок, будущего кода. Только база гарантирует уникальность при параллельных запросах.

```
# вход API
class ReactionInSchema(BaseModel):
    value: Literal[1, -1]

# данные
class ArticleReaction(Base):
    value: Mapped[int] = mapped_column(SmallInteger)
    __table_args__ = (CheckConstraint("value IN (-1, 1)", name="ck_article_reactions_value"),)
```

49. Как бороться с гонками при параллельных запросах? → глава 11

Не проверять «есть ли уже» в коде, а полагаться на уникальный ключ и ловить `IntegrityError` или `ON CONFLICT`. Счётчики менять выражением в базе. Транзакции короткие. Операции идемпотентные.

```
# плохо: между чтением и записью другой запрос успеет
article.views_count = article.views_count + 1

# хорошо: арифметика в базе под блокировкой строки
await db.execute(
    update(Article).where(Article.id == article_id)
    .values(views_count=Article.views_count + 1)
)
# идемпотентность: повторный лайк тем же значением ничего не меняет
if reaction is not None and reaction.value == value:
    return
```

50. Как вы проектируете схему под новую функцию? → глава 46

Сущности и связи словами → для каждой колонки тип, обязательность, уникальность, ограничения, индексы под известные запросы → правила удаления → модель → миграция → сервис → схемы API → тест. Раздел «Новости» потребовал одну колонку, а не вторую таблицу.

```
# 1. модель
section: Mapped[str] = mapped_column(String(16), default="blog", server_default="blog", index=True)
# 2. миграция 0004: add_column + create_index
```

```
# 3. сервис: фильтр
if section:
    stmt = stmt.where(Article.section == section)
# 4. API: Query-параметр с Literal
section: Section | None = Query(None)
# 5. тест
r = client.get("/api/v1/articles?section=news")
assert all(a["section"] == "news" for a in r.json()["articles"])
```

Как готовиться. Не зубрите ответы. По каждому вопросу откройте место в своём коде, где это применено, и объясните вслух, почему сделано именно так. Интервьюер почти всегда уходит с вопроса на проект: «а у вас как?» — и это лучший момент собеседования.